

PROPOSAL TUGAS AKHIR

**PERBANDINGAN TERKONTROL PARADIGMA *DEPLOYMENT*
MANUAL DAN GITOPS DALAM MENGHADAPI PENYIMPANGAN
KONFIGURASI PADA KLASER KUBERNETES**

***A CONTROLLED COMPARISON OF MANUAL AND GITOPS
DEPLOYMENT PARADIGMS IN RESPONDING TO
CONFIGURATION DRIFT ON KUBERNETES CLUSTERS***



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

PROPOSAL TUGAS AKHIR

**PERBANDINGAN TERKONTROL PARADIGMA *DEPLOYMENT*
MANUAL DAN GITOPS DALAM MENGHADAPI PENYIMPANGAN
KONFIGURASI PADA KLASSTER KUBERNETES**

***A CONTROLLED COMPARISON OF MANUAL AND GITOPS
DEPLOYMENT PARADIGMS IN RESPONDING TO
CONFIGURATION DRIFT ON KUBERNETES CLUSTERS***

Usulan Penelitian



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

HALAMAN PENGESAHAN

PROPOSAL TUGAS AKHIR

PERBANDINGAN TERKONTROL PARADIGMA *DEPLOYMENT* MANUAL DAN GITOPS DALAM MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

Telah dipersiapkan dan disusun oleh

MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

Telah dipertahankan di depan Tim Penguji
pada tanggal 17 Juni 2026

Susunan Tim Penguji

Guntur Budi Herwanto, S.Kom., M.Cs. Pembimbing	Dr. techn. Kabul Kurniawan, S.Kom., M.Cs. Ketua Penguji
--	---

Aina Musdholifah, S.Kom., M.Kom.,
Ph.D.
Anggota Penguji

DAFTAR ISI

Halaman Judul	ii
Halaman Pengesahan	iii
INTISARI	vii
ABSTRACT	viii
I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Batasan Masalah	4
1.4 Tujuan Penelitian	5
1.5 Manfaat Penelitian	5
1.6 Sistematika Penulisan	6
II PENELITIAN TERKAIT	7
2.1 Penyimpangan Konfigurasi (<i>Configuration Drift</i>)	7
2.1.1 Faktor yang Menyebabkan Penyimpangan Konfigurasi	7
2.2 Terminologi Dasar	8
2.2.1 Kubernetes dan Orkestrasi Kontainer	8
2.2.2 Git dan <i>Version Control</i>	8
2.2.3 <i>Continuous Integration</i> dan <i>Continuous Deployment</i>	9
2.2.4 <i>Infrastructure-as-Code</i> (IaC)	9
2.2.5 Paradigma Deklaratif vs Imperatif	9
2.3 Manajemen Konfigurasi Infrastruktur	9
2.4 <i>Continuous Deployment</i> dan Paradigma GitOps	10
2.4.1 Performa <i>Deployment</i> dan <i>Anti-Pattern</i>	10
2.5 Dasar Teori: <i>Reconciliation</i> dan <i>Convergence</i> pada Sistem Deklaratif .	11
2.5.1 <i>Convergence</i> Menuju <i>Desired State</i>	11
2.5.2 <i>Audit Trail</i> sebagai <i>Social Control Mechanism</i>	12
2.6 <i>Runbook</i> sebagai Standarisasi Prosedur Pemulihan	12
2.7 Paradigma <i>Deployment</i> Alternatif	13
2.8 Evaluasi Empiris GitOps dan ArgoCD	13

2.9	Metode Eksperimental Terkontrol pada Riset Sistem	15
2.10	Klasifikasi Beban Uji (<i>Workload</i>)	15
2.11	Kesenjangan Penelitian (<i>Research Gap</i>)	16

III METODE DAN RANCANGAN PENELITIAN **18**

3.1	Deskripsi Umum Penelitian	18
3.2	Metodologi	20
3.3	Definisi Intervensi dan Variabel Penelitian	21
3.3.1	Intervensi (Perlakuan)	21
3.3.2	Variabel Independen (Variabel Bebas)	22
3.3.3	Variabel Dependen (Variabel Terikat)	22
3.4	Justifikasi Ekuitas Perbandingan (<i>Apple-to-Apple</i>)	24
3.5	Peran Peneliti dan Standarisasi Prosedur	25
3.5.1	Mitigasi Bias Keahlian <i>Operator</i> pada Waktu Identifikasi Akar Masalah	26
3.6	Skrip Otomasi Eksperimen	27
3.6.1	Skrip Injeksi Penyimpangan (<i>drift-inject.sh</i>)	27
3.6.2	Skrip Pengumpulan Metrik (<i>metrics-collect.sh</i>)	28
3.6.3	Skrip Verifikasi <i>Baseline</i> (<i>baseline-check.sh</i>)	28
3.6.4	Skrip <i>Recovery Runbook</i> (<i>recovery-runbook.sh</i>)	28
3.7	Lingkungan Eksperimen	29
3.7.1	Klaster Kubernetes	29
3.7.2	ArgoCD (Lingkungan B)	29
3.7.3	Beban Uji (<i>Workload</i>)	30
3.7.4	Konfigurasi dan Dataset	30
3.7.5	Kontrol Variabel Pengganggu	30
3.7.6	Protokol <i>Monitoring</i> Variabel Terkontrol	31
3.8	Analisis Power dan Justifikasi Ukuran Sampel	31
3.9	Matriks Eksperimen	32
3.10	Skenario Penyimpangan Terkontrol	32
3.10.1	Skenario A: Perubahan Replika (<i>Replica Drift</i>)	33
3.10.2	Skenario B: Perubahan <i>Image</i> (<i>Image Drift</i>)	33
3.10.3	Skenario C: Perubahan ConfigMap (<i>ConfigMap Drift</i>)	33
3.10.4	Skenario D: Penghapusan Sumber Daya (<i>Resource Deletion</i>)	33

3.10.5	Skenario E: <i>Patch</i> Manual Tidak Sah (<i>Unauthorized Manual Patch</i>)	33
3.10.6	Skenario F: Penyimpangan Lintas <i>Namespace</i> (<i>Cross-namespace Drift</i>)	33
3.10.7	Skenario G: Perubahan Secret (<i>Secret Drift</i>)	34
3.11	Protokol Pengukuran	34
3.11.1	Protokol Pengukuran <i>Baseline</i> (O1)	34
3.11.2	Protokol Injeksi Penyimpangan (X)	34
3.11.3	Protokol Pengukuran Pasca-Injeksi (O2)	34
3.12	Metode Pengujian (Eksperimen E1 s.d. E3)	35
3.12.1	E1: Konsistensi Deployment (R1)	35
3.12.2	E2: Waktu Deteksi, Pemulihan, dan Identifikasi Akar Masalah (R2)	36
3.12.3	E3: <i>Traceability</i> Operasional (R3, Deskriptif)	36
3.13	Analisis Statistik Komparatif	36
3.14	Ancaman terhadap Validitas	37
3.14.1	Validitas Internal	37
3.14.2	Validitas Eksternal	38
3.14.3	Validitas Konstruksi	38
3.14.4	Reliabilitas	39
IV	JADWAL PENELITIAN	40
4.1	Penjelasan Kegiatan	40

INTISARI

Perbandingan Terkontrol Paradigma *Deployment* Manual dan GitOps dalam Menghadapi Penyimpangan Konfigurasi pada Klaster Kubernetes

Oleh

Muhammad Argya Vityasy

23/522547/PA/22475

Penyimpangan konfigurasi (*configuration drift*) pada klaster Kubernetes merupakan masalah operasional yang signifikan: *state* aktual klaster menyimpang dari *state* yang didefinisikan secara deklaratif, mengakibatkan inkonsistensi, *audit trail* yang buruk, dan waktu pemulihan yang lama. Walaupun paradigma GitOps secara intuitif diyakini lebih unggul dalam menghadapi penyimpangan, bukti empiris terkontrol yang mengkarakterisasi besarnya dan kondisi perbedaan antara paradigma manual dan GitOps belum tersedia. Penelitian yang diuraikan dalam proposal ini bertujuan mengusulkan pendekatan berbasis GitOps untuk menghadapi penyimpangan konfigurasi pada *deployment* multi-service di klaster Kubernetes, dengan pencapaiannya didemonstrasikan melalui evaluasi empiris terkontrol. Evaluasi menggunakan desain eksperimental dengan dua lingkungan paralel: Lingkungan A (*deployment* manual) dan Lingkungan B (*deployment* GitOps). Pada kedua lingkungan, penyimpangan konfigurasi di-*inject* secara terkontrol melalui tujuh skenario (perubahan replika, perubahan *image*, modifikasi ConfigMap, *resource deletion*, *patch* manual, penyimpangan lintas *namespace*, dan perubahan Secret). Pengukuran dilakukan terhadap tiga metrik utama: (1) konsistensi *deployment*; (2) waktu deteksi, waktu pemulihan, dan waktu identifikasi akar masalah; dan (3) *traceability* operasional, serta beban kognitif *operator* (NASA-TLX) sebagai *measured confound*. Setiap skenario diulang minimal lima kali per lingkungan untuk menghasilkan data yang memadai. Hasil dianalisis menggunakan uji statistik komparatif (*Mann–Whitney U*) dan ukuran efek (Cohen’s d atau Cliff’s δ) untuk mengkarakterisasi besarnya perbedaan antara kedua paradigma pada berbagai skenario penyimpangan.

Kata Kunci: penyimpangan konfigurasi, Kubernetes, GitOps, *deployment* manual, evaluasi empiris terkontrol

ABSTRACT

A Controlled Comparison of Manual and GitOps Deployment Paradigms in Responding to Configuration Drift on Kubernetes Clusters

By

Muhammad Argya Vityasy

23/522547/PA/22475

Configuration drift on Kubernetes clusters is a significant operational problem: the actual cluster state deviates from the declaratively defined state, resulting in inconsistency, poor audit trails, and lengthy recovery times. Although GitOps is intuitively believed to be superior in handling drift, controlled empirical evidence characterizing the magnitude and conditions of the difference between manual and GitOps paradigms is lacking. The research described in this proposal aims to propose a GitOps-based approach for handling configuration drift in multi-service deployment on Kubernetes clusters, with its achievement demonstrated through a controlled empirical evaluation. The study adopts a controlled experimental design with two parallel environments: Environment A (manual deployment) and Environment B (GitOps deployment). On both environments, controlled configuration drift is injected through seven scenarios (replica change, image change, ConfigMap modification, resource deletion, manual patch, cross-namespace drift, and Secret change). Measurements are taken across three primary metrics: (1) deployment consistency; (2) drift detection time, recovery time, and root cause identification time; and (3) operational traceability, with operator cognitive load (NASA-TLX) as a measured confound. Each scenario is repeated at least five times per environment to yield sufficient data. Results are analyzed using comparative statistical tests (Mann–Whitney U) and effect sizes (Cohen’s d or Cliff’s delta) to characterize the magnitude of differences between the two paradigms across various drift scenarios.

Keywords: configuration drift, Kubernetes, GitOps, manual deployment, controlled empirical evaluation

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kubernetes telah menjadi platform *container orchestration* yang dominan di industri maupun akademisi, digunakan oleh lebih dari 5,6 juta *developer* secara global (Cloud Native Computing Foundation, 2024). Kemampuan Kubernetes dalam mengelola *deployment*, *scaling*, dan *self-healing* menjadikannya pilihan utama untuk menjalankan aplikasi yang memerlukan *availability* tinggi (Burns et al., 2016). Namun, seiring bertambahnya jumlah layanan yang di-*deploy* pada sebuah kluster, kompleksitas operasional meningkat secara drastis. Aplikasi yang dijalankan pada Kubernetes mencakup berbagai kelas *workload* dengan karakteristik yang berbeda, antara lain: (1) *workload* web tanpa state (*stateless web microservice*), (2) *workload* berbasis data yang menyimpan state (*stateful data service*), serta (3) *workload* komposit multi-service yang terdiri atas banyak komponen yang saling bergantung (Siddiqui et al., 2023; Aghili et al., 2024). Kelas terakhir, seperti platform data repositori skala besar, dapat terdiri dari 16 atau lebih komponen: layanan *web*, *worker* asinkron, *database* relasional, *search engine*, *cache*, *object storage*, *ingress controller*, *certificate manager*, enkripsi *secret*, *tunnel* jaringan, kebijakan keamanan, *monitoring system*, dan *backup*.

Pengelolaan *deployment* aplikasi multi-service menggunakan pendekatan manual, yakni melalui perintah `kubectl apply`, *manual scaling*, atau *patching* langsung pada kluster, sering menghasilkan inkonsistensi operasional. Praktik ini rentan terhadap penyimpangan konfigurasi (*configuration drift*), yaitu situasi di mana *state* aktual kluster menyimpang dari *state* yang didefinisikan secara deklaratif (Zhang et al., 2026). Penyimpangan konfigurasi dapat timbul akibat tiga penyebab utama: (1) perubahan langsung pada kluster yang tidak tercatat dalam sistem *version control*; (2) prosedur *deployment* yang tidak konsisten antar *operator*; dan (3) ketiadaan mekanisme deteksi otomatis yang dapat mengidentifikasi deviasi sebelum menimbulkan insiden.

Penelitian empiris oleh Zhang et al. (2026) menunjukkan bahwa 533 dari 719 *defect* (74,1%) menyebabkan *crash*, operasi yang salah, atau *outage*. Rahman et al. (2023) menambahkan bahwa *security misconfiguration* pada manifest Kuberne-

tes mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai *root* dan tanpa *resource limits*. Studi-studi tersebut mengonfirmasi bahwa penyimpangan konfigurasi bukan sekadar masalah estetika konfigurasi, melainkan ancaman nyata terhadap keandalan dan keamanan layanan.

GitOps telah muncul sebagai paradigma yang diajukan sebagai alternatif untuk mengatasi masalah tersebut dengan menjadikan repositori Git sebagai *single source of truth* dan mengandalkan agen *software* untuk menyelaraskan *state* kluster secara otomatis (CNCF GitOps Working Group, 2021). Beberapa penelitian mendukung klaim ini secara kualitatif: Nataraja (2025) menemukan bahwa pendekatan deklaratif menghasilkan koreksi penyimpangan yang lebih cepat dan paparan kerentanan yang lebih rendah; Shrestha and Ali (2024) melaporkan karakteristik GitOps pada aspek deteksi penyimpangan; dan Matubber (2025) mengukur waktu *reconciliation* GitOps terhadap *self-healing*. Namun, temuan-temuan tersebut mengandung kelemahan metodologis, yakni tidak ada yang mengisolasi efek paradigma *deployment* dari variabel pengganggu, dan tidak ada yang menerapkan uji statistik formal pada perbandingan terkontrol (Shrestha and Ali, 2024; Nataraja, 2025; Matubber, 2025).

Walaupun secara intuitif paradigma GitOps diyakini lebih unggul, perbandingan terkontrol tetap bermakna karena tiga alasan. Pertama, tanpa bukti empiris terkontrol, klaim unggulnya GitOps tidak dapat diatribusikan secara kausal kepada paradigma itu sendiri; temuan-temuan yang ada mungkin disebabkan oleh variabel pengganggu seperti keahlian *operator*, karakteristik kluster, atau kompleksitas *workload*. Kedua, *magnitude* perbedaan antara kedua paradigma belum terkuantifikasi, mengetahui bahwa GitOps lebih baik secara kualitatif tidak memberikan informasi tentang seberapa jauh lebih baik, pada skenario apa, dan dalam kondisi apa. Ketiga, pemahaman tentang kondisi di mana *manual deployment* tetap memadai (misalnya, pada kluster kecil dengan perubahan jarang) memerlukan data komparatif yang terukur (Forsgren et al., 2018).

Kesenjangan tersebut menjadi landasan penelitian yang diuraikan dalam proposal ini: terdapat kebutuhan untuk mengevaluasi secara empiris seberapa besar dampak penyimpangan konfigurasi terhadap karakteristik operasional *deployment* (konsistensi, waktu deteksi/pemulihan, dan *traceability*) ketika berbagai skenario penyimpangan terjadi pada dua paradigma manajemen *deployment* yang berbeda, yaitu manual dan GitOps, melalui evaluasi empiris terkontrol pada *workload* multi-service di kluster Kubernetes. Hasil evaluasi tersebut mengkarakterisasi besarnya dan kondisi perbedaan antara kedua paradigma, yang selanjutnya menjadi dasar rekomendasi

pemilihan paradigma *deployment* bagi praktisi industri.

1.2 Rumusan Masalah

Pengelolaan *deployment* aplikasi multi-service pada kluster Kubernetes menghadapi tiga permasalahan praktis yang mendorong perlunya evaluasi paradigma *deployment*. Pertama, inkonsistensi konfigurasi: perubahan langsung pada kluster yang tidak tercatat dalam *version control*, prosedur *deployment* yang tidak konsisten antar *operator*, dan ketiadaan mekanisme deteksi otomatis menyebabkan *state* aktual kluster menyimpang dari *state* yang dideklarasikan, dengan konsekuensi yang meliputi *crash*, operasi yang salah, atau *outage* (Zhang et al., 2026; Rahman et al., 2023). Kedua, lamanya waktu yang diperlukan untuk mendeteksi dan mengidentifikasi akar masalah: tanpa mekanisme *reconciliation* otomatis, *operator* harus memeriksa log dan manifest secara manual, sehingga interval antara terjadinya penyimpangan dan pemulihannya bergantung sepenuhnya pada kewaspadaan dan keahlian *operator* (Pohjola, 2025). Ketiga, minimnya *traceability* operasional: *deployment* manual tidak menyediakan jejak perubahan yang terpusat, sehingga rekonstruksi siapa yang mengubah *state*, apa yang berubah, dan kapan perubahan terjadi menjadi sulit atau tidak mungkin dilakukan.

Paradigma GitOps diajukan sebagai alternatif yang berpotensi mengatasi permasalahan-permasalahan tersebut melalui *reconciliation* otomatis dan *audit trail* bawaan. Namun, studi-studi terdahulu yang mendukung GitOps tidak mengisolasi efek paradigma *deployment* dari variabel pengganggu seperti keahlian *operator*, karakteristik kluster, dan jenis *workload*, sehingga klaim unggulnya GitOps belum dapat diatribusikan secara kausal kepada paradigma itu sendiri (Shrestha and Ali, 2024; Nataraja, 2025; Matubber, 2025). Akibatnya, praktisi industri belum memiliki dasar berbasis bukti untuk memilih paradigma *deployment* yang sesuai. Kesenjangan ini menjadi landasan bagi penelitian yang diuraikan dalam proposal ini: terdapat kebutuhan untuk mengevaluasi secara empiris seberapa besar dampak penyimpangan konfigurasi terhadap karakteristik operasional *deployment* ketika berbagai skenario penyimpangan terjadi pada dua paradigma manajemen *deployment* yang berbeda, yaitu manual dan GitOps, melalui evaluasi empiris terkontrol pada *workload* multi-service di kluster Kubernetes.

Evaluasi tersebut mencakup tiga dimensi pengukuran. Pertama, sejauh mana konsistensi *deployment* berbeda antara kedua paradigma setelah terjadinya penyim-

pangan konfigurasi. Kedua, seberapa besar perbedaan waktu deteksi, waktu pemulihan, dan waktu identifikasi akar masalah antara lingkungan yang mendeteksi dan memulihkan secara otomatis dan lingkungan yang bergantung pada intervensi manual. Ketiga, sejauh mana perbedaan *traceability* operasional antara paradigma yang mencatat setiap perubahan secara otomatis melalui Git dan paradigma yang mengandalkan pencatatan manual. Ketiga dimensi diuji terhadap tujuh skenario penyimpangan konfigurasi yang merepresentasikan kesalahan operasional umum pada Kubernetes: (A) perubahan replika yang melewati deklarasi Git, (B) perubahan *image* kontainer langsung pada kluster, (C) modifikasi ConfigMap tanpa pembaruan *source of truth*, (D) penghapusan sumber daya secara manual, (E) *patch* yang tidak sah pada sumber daya yang dikelola, (F) aplikasi sumber daya pada *namespace* yang salah, dan (G) perubahan Secret di luar Git.

1.3 Batasan Masalah

Agar penelitian tetap fokus dan terukur, batasan masalah yang ditetapkan adalah:

1. Penelitian dilakukan pada kluster Kubernetes tunggal yang dikelola Rancher dengan spesifikasi 3 *node*, 24 total inti CPU, dan ~23 Gi total memori; skenario *multi-cluster* tidak termasuk dalam ruang lingkup.
2. Kelas *workload* yang digunakan adalah *workload* komposit multi-service, yang mencakup setidaknya 16 komponen infrastruktur dan dependensi termasuk layanan *web*, *worker* asinkron, *database* relasional, *search engine*, *cache*, *object storage*, *ingress controller*, *certificate manager*, dan *monitoring system*. Temuan berlaku untuk kelas *workload* multi-service secara umum.
3. Skenario penyimpangan konfigurasi yang diuji terbatas pada tujuh kategori: (A) perubahan replika, (B) perubahan *image*, (C) modifikasi ConfigMap, (D) *resource deletion*, (E) *patch* manual tidak sah, (F) penyimpangan lintas *namespace*, dan (G) perubahan Secret.
4. Dua paradigma manajemen *deployment* dibandingkan: (a) *deployment* manual menggunakan `kubectl`, dan (b) *deployment* GitOps menggunakan ArgoCD sebagai salah satu implementasi paradigma GitOps.

5. Analisis statistik menggunakan *Mann–Whitney U test* untuk uji signifikansi perbedaan antara kedua lingkungan, dengan tingkat signifikansi $\alpha = 0,05$ dan ukuran efek (Cohen’s d atau Cliff’s δ).
6. Paradigma perantara (Git sebagai *source of truth* tanpa agen *reconciliation* otomatis) dan paradigma *push-based* (seperti Jenkins CI/CD atau Ansible yang mendorong perubahan ke klaster dari luar) tidak termasuk dalam ruang lingkup. Paradigma perantara tidak memiliki mekanisme koreksi otomatis sehingga tidak memenuhi prinsip GitOps *continuous reconciliation*, sementara paradigma *push-based* memerlukan kredensial klaster yang terekspos dan tidak memiliki *feedback loop* bawaan (Utami and Fatta, 2021; Shrestha and Ali, 2024). Keduanya merepresentasikan titik yang berbeda pada spektrum deklaratif-imperatif dan dapat dievaluasi pada penelitian lanjutan.

1.4 Tujuan Penelitian

Tujuan penelitian yang diuraikan dalam proposal ini adalah mengusulkan pendekatan berbasis GitOps untuk menghadapi penyimpangan konfigurasi pada *deployment* multi-service di klaster Kubernetes, dengan pencapaiannya didemonstrasikan melalui pengukuran kinerja terkontrol yang mencakup: (1) konsistensi *deployment* setelah skenario penyimpangan terkontrol, (2) waktu deteksi, waktu pemulihan, dan waktu identifikasi akar masalah, serta (3) *traceability* operasional.

1.5 Manfaat Penelitian

Penelitian yang diuraikan dalam proposal ini diharapkan memberikan manfaat berikut:

1. Teoritis: Memberikan bukti empiris terukur mengenai dampak operasional skenario penyimpangan konfigurasi pada dua paradigma *deployment*, yang selama ini banyak didasarkan pada klaim *vendor* atau studi kasus deskriptif. Hasil tersebut melengkapi literatur akademis dengan data eksperimental yang dapat direproduksi dan diverifikasi.
2. Metodologis: Menawarkan protokol eksperimental yang dapat direplikasi, termasuk skenario penyimpangan terstandarisasi, metrik operasional, dan prose-

dur analisis statistik, sehingga penelitian selanjutnya dapat mereplikasi, memperluas, atau mengkritik temuan pada konteks dan platform yang berbeda.

3. **Praktis:** Menyediakan bukti empiris terkontrol yang mengkarakterisasi perbedaan operasional antara kedua paradigma *deployment* pada berbagai skenario penyimpangan, serta menyoroti skenario penyimpangan yang paling rentan pada masing-masing paradigma, sehingga membantu praktisi industri dalam memilih paradigma *deployment* yang sesuai.

1.6 Sistematika Penulisan

Sistematika penulisan proposal tugas akhir ini adalah sebagai berikut:

- Bab I: Pendahuluan, berisi latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, dan sistematika penulisan.
- Bab II: Penelitian Terkait, membahas studi empiris terkait penyimpangan konfigurasi pada Kubernetes, terminologi dasar, manajemen konfigurasi infrastruktur, paradigma *deployment* alternatif, evaluasi GitOps dan ArgoCD, metode eksperimental pada riset sistem, serta *research gap* yang menjadi landasan penelitian.
- Bab III: Metode dan Rancangan Penelitian, menjelaskan metodologi evaluasi empiris terkontrol, definisi variabel dan intervensi, tujuh skenario penyimpangan, protokol pengukuran, analisis statistik komparatif, ancaman terhadap validitas, dan lingkungan eksperimen.
- Bab IV: Jadwal Penelitian, memuat Gantt Chart berbasis bulan untuk perencanaan waktu penelitian selama 6 bulan.

BAB II

PENELITIAN TERKAIT

2.1 Penyimpangan Konfigurasi (*Configuration Drift*)

Penyimpangan konfigurasi (*configuration drift*) adalah situasi di mana *state* aktual sebuah sistem menyimpang dari *state* yang didefinisikan secara deklaratif (Pohjola, 2025). Dalam konteks Kubernetes, penyimpangan terjadi ketika *operator*, baik secara langsung maupun tidak sengaja, mengubah manifest kluster tanpa memperbarui *source of truth* deklaratif, sehingga *state* aktual dan *desired state* menjadi tidak konsisten.

Penelitian empiris oleh Zhang et al. (2026) melakukan studi terhadap 719 *defect* konfigurasi dari 2.260 skrip Kubernetes dan mengidentifikasi 15 kategori *defect*, di mana 533 dari 719 *defect* (74,1%) menyebabkan *crash*, operasi yang salah, atau *outage*. Studi tersebut menegaskan bahwa *misconfiguration* merupakan risiko operasional utama pada Kubernetes. Rahman et al. (2023) menambahkan bahwa *security misconfiguration* pada manifest Kubernetes *open-source* mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai *root*, tanpa *drop capabilities*, dan tanpa *resource limits*. Penelitian menunjukkan bahwa penyimpangan konfigurasi bukan hanya masalah estetika konfigurasi, tetapi ancaman nyata terhadap keandalan dan keamanan layanan.

Pohjola (2025) memberikan kerangka konseptual untuk memahami sumber dan dampak penyimpangan pada sistem *Infrastructure-as-Code* (IaC). Penelitian tersebut mengidentifikasi tiga faktor pendorong utama penyimpangan: (1) keahlian manusia (*human expertise*) yang tidak konsisten; (2) tekanan waktu (*time pressure*) yang memicu perubahan cepat dan tidak didokumentasikan; dan (3) ketiadaan mekanisme deteksi otomatis pada *pipeline deployment*. Meskipun penelitian tersebut tidak beroperasi pada Kubernetes secara eksklusif, prinsip-prinsip yang dikemukakan berlaku umum untuk platform orkestrasi berbasis konfigurasi deklaratif.

2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi

Berdasarkan literatur, faktor-faktor utama yang memperburuk penyimpangan konfigurasi pada Kubernetes meliputi:

1. Perubahan langsung pada kluster: Penggunaan `kubectl edit`, `kubectl`

patch, atau `kubectl scale` secara langsung tanpa memperbarui repositori *source of truth* (Zhang et al., 2026).

2. Prosedur *deployment* yang tidak terdokumentasi: Setiap *operator* dapat memiliki urutan perintah yang berbeda, yang mengakibatkan *state* kluster tidak deterministik (Pohjola, 2025).
3. Kurangnya mekanisme *rollback* yang terintegrasi: *Deployment* manual seringkali tidak memiliki mekanisme *rollback* otomatis; *operator* harus mengingat konfigurasi sebelumnya atau mengandalkan *backup* yang mungkin tidak konsisten (Rahman et al., 2023).
4. Ketiadaan mekanisme deteksi penyimpangan otomatis: Tanpa pemeriksaan berkala antara *state* aktual dan *desired state*, penyimpangan dapat terakumulasi selama periode lama tanpa terdeteksi (Zhang et al., 2026).

2.2 Terminologi Dasar

Bagian ini mendefinisikan terminologi kunci yang digunakan dalam penelitian, dengan tujuan membangun dasar konseptual yang konsisten sebelum pembahasan lebih lanjut.

2.2.1 Kubernetes dan Orkestrasi Kontainer

Kubernetes adalah platform *container orchestration open-source* yang mengotomasi *deployment*, *scaling*, dan manajemen aplikasi terkemas (*containerized*) (Burns et al., 2016). Kubernetes awalnya dikembangkan oleh Google berdasarkan pengalaman mengelola sistem *workload* berskala besar di Borg dan Omega, dan kini menjadi standar *de facto* untuk orkestrasi kontainer di industri (Verma et al., 2015). Pada Kubernetes, *state* yang diinginkan (*desired state*) dideklarasikan melalui berkas konfigurasi (*manifest*), dan *control plane* bertanggung jawab mencapai dan mempertahankan *state* tersebut.

2.2.2 Git dan Version Control

Git adalah sistem *version control* terdistribusi yang merekam setiap perubahan pada berkas beserta metadata (penulis, *timestamp*, pesan *commit*) (Chacon and Straub, 2014). Dalam konteks GitOps, Git berfungsi sebagai *single source of truth*

yang menyimpan deklarasi *state* infrastruktur dan aplikasi, sehingga setiap perubahan dapat dilacak, di-review, dan dikembalikan (*rollback*) secara andal.

2.2.3 *Continuous Integration dan Continuous Deployment*

Continuous Integration (CI) adalah praktik di mana perubahan kode secara otomatis diuji dan digabungkan ke *branch* utama, sementara *Continuous Deployment* (CD) adalah perluasan CI di mana perubahan yang lulus pengujian secara otomatis di-deploy ke lingkungan produksi tanpa intervensi manual eksplisit (Humble and Farley, 2010). GitOps merupakan paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth*.

2.2.4 *Infrastructure-as-Code (IaC)*

Infrastructure-as-Code (IaC) adalah praktik mendefinisikan dan mengelola infrastruktur melalui berkas konfigurasi yang diberi versi (*versioned*), bukan melalui proses manual (Morris, 2020). IaC memungkinkan *version control*, *code review*, dan reproduktibilitas lingkungan. Dalam konteks Kubernetes, IaC diimplementasikan melalui *manifest* deklaratif (YAML) yang mendefinisikan *desired state* kluster.

2.2.5 *Paradigma Deklaratif vs Imperatif*

Dalam paradigma imperatif, *operator* memberikan serangkaian perintah langkah demi langkah yang mengubah *state* sistem secara langsung (*how to achieve the state*). Dalam paradigma deklaratif, *operator* mendefinisikan *desired state* dan sistem bertanggung jawab mencapai serta mempertahankan *state* tersebut (*what state to achieve*) (Scott, 2015). Perbedaan fundamental ini memiliki implikasi langsung terhadap bagaimana sistem merespons deviasi konfigurasi: paradigma deklaratif mendukung *convergence* otomatis, sementara paradigma imperatif memerlukan intervensi manual eksplisit untuk setiap deviasi.

2.3 *Manajemen Konfigurasi Infrastruktur*

Infrastructure state management adalah disiplin yang mengatur bagaimana *state* sebuah sistem infrastruktur didefinisikan, diterapkan, dan dipertahankan. Dalam paradigma deklaratif, *operator* mendefinisikan *desired state* melalui konfigurasi

yang telah dibakukan (*Infrastructure-as-Code*), dan sistem bertanggung jawab untuk mencapai serta mempertahankan *state* tersebut (Morris, 2020). Dalam paradigma imperatif, *operator* memberikan serangkaian perintah langkah demi langkah yang mengubah *state* sistem secara langsung (Limoncelli et al., 2018). Perbedaan fundamental ini memiliki implikasi langsung terhadap bagaimana sistem merespons deviasi konfigurasi: paradigma deklaratif mendukung *convergence* otomatis, sementara paradigma imperatif memerlukan intervensi manual untuk setiap deviasi.

2.4 Continuous Deployment dan Paradigma GitOps

Continuous Deployment (CD) adalah praktik dalam ranah DevOps di mana setiap perubahan kode yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Limoncelli et al., 2018). CD merupakan perluasan dari *Continuous Integration* (CI) dan bertujuan meminimalkan waktu antara penulisan kode dan ketersediaannya bagi pengguna akhir. Dalam ekosistem Kubernetes, CD memerlukan mekanisme yang dapat secara andal menyelaraskan perubahan konfigurasi dengan *state* kluster.

GitOps adalah paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth* untuk mendefinisikan *state* infrastruktur dan aplikasi. CNCF GitOps Working Group mendefinisikan empat prinsip OpenGitOps: (1) sistem dideklarasikan secara deklaratif; (2) *state* sistem yang diinginkan tersimpan dan diberi versi (*versioned*) dalam Git; (3) perubahan *state* ditarik (*pulled*) secara otomatis oleh agen *software*; dan (4) agen *software* secara kontinu mendeteksi dan mengoreksi deviasi dari *state* yang diinginkan (CNCF GitOps Working Group, 2021).

Karakteristik GitOps yang berbeda dari pendekatan imperatif tradisional meliputi: *audit trail* lengkap melalui Git *history*, penerapan *code review* pada perubahan infrastruktur, *rollback* yang sederhana melalui `git revert`, dan reproduisibilitas konfigurasi di berbagai lingkungan (Yuen et al., 2021). Dalam model *pull-based* GitOps, agen yang berjalan di dalam kluster secara periodik membandingkan *state* Git dengan *state* kluster dan menyelaraskan keduanya, menghilangkan kebutuhan akan akses kredensial kluster dari luar (Utami and Fatta, 2021).

2.4.1 Performa Deployment dan Anti-Pattern

Forsgren dkk. (Forsgren et al., 2018) mengidentifikasi empat metrik kunci yang mengkarakterisasi performa *deployment* pada organisasi teknologi: (1) *depl-*

ymment frequency, (2) *lead time for changes*, (3) *time to restore service*, dan (4) *change failure rate*. Metrik-metrik tersebut menunjukkan bahwa organisasi yang mengadopsi praktik *continuous deployment* dan *Infrastructure-as-Code* secara konsisten mengungguli organisasi yang bergantung pada proses manual. Lebih lanjut, praktik *deployment* manual yang mengabaikan *version control* dan *automated reconciliation* merupakan *anti-pattern* yang berkontribusi terhadap penyimpangan konfigurasi (Beyer et al., 2016). Dalam konteks penelitian yang diuraikan dalam proposal ini, metrik *time to restore service* dan *change failure rate* berelasi langsung dengan metrik R2 (waktu deteksi/pemulihan) dan R1 (konsistensi *deployment*) yang digunakan dalam eksperimen.

ArgoCD mengimplementasikan prinsip-prinsip GitOps melalui *reconciliation loop* yang berjalan di dalam kluster. ArgoCD terdiri dari tiga komponen utama: (1) *application controller* yang secara periodik membandingkan *state* Git dengan *state* aktual dan melakukan tindakan korektif; (2) *repo server* yang mengelola koneksi ke repositori Git; dan (3) *Redis cache* yang menyimpan *state* sementara untuk performa (Argo Project, 2024). ArgoCD berperan sebagai *negative feedback controller*: setiap deviasi (penyimpangan) bersifat *transien*, yaitu sistem secara otomatis kembali ke *desired state* tanpa memerlukan intervensi manusia, sepanjang *self-heal* diaktifkan.

2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif

Karakteristik GitOps yang berbeda dari pendekatan imperatif dapat dijelaskan melalui dua prinsip teoretis yang saling berkaitan.

2.5.1 *Convergence Menuju Desired State*

Dalam teori sistem, konsep *negative feedback controller* menjelaskan bagaimana sistem yang menyimpan *desired state* dan secara kontinu berupaya mencapainya dikatakan bersifat *convergent* (Limoncelli et al., 2018). ArgoCD mengimplementasikan prinsip ini melalui *reconciliation loop*: secara periodik (1) membaca *desired state* dari repositori Git, (2) membandingkannya dengan *state* aktual di kluster, dan (3) melakukan tindakan korektif jika terdapat deviasi (Argo Project, 2024). Mekanisme ini menjamin bahwa setiap deviasi (penyimpangan) bersifat *transien*, yakni sistem secara otomatis kembali ke *desired state* tanpa memerlukan intervensi manusia, sepanjang *self-heal* diaktifkan.

Sebaliknya, pendekatan manual/imperatif tidak memiliki *feedback loop* bawaan. Setiap deviasi bersifat *persisten* sampai *operator* secara manual mendeteksi dan mengoreksinya. Tanpa mekanisme otomatis, interval antara penyimpangan dan pemulihan bergantung sepenuhnya pada kewaspadaan dan ketersediaan *operator*, yang bervariasi secara tidak deterministik (Pohjola, 2025).

2.5.2 *Audit Trail sebagai Social Control Mechanism*

GitOps memanfaatkan Git sebagai *append-only log* yang merekam setiap perubahan konfigurasi beserta metadata (penulis, waktu, pesan *commit*). Prinsip keempat OpenGitOps, yaitu *continuous reconciliation*, bergantung pada kemampuan Git untuk merekonstruksi keputusan konfigurasi secara retroaktif (CNCF GitOps Working Group, 2021). Hal tersebut tidak hanya meningkatkan *traceability*, tetapi juga berfungsi sebagai *social control mechanism*: setiap perubahan tunduk pada *code review*, sehingga mengurangi kemungkinan perubahan yang tidak terverifikasi masuk ke produksi.

2.6 *Runbook sebagai Standarisasi Prosedur Pemulihan*

Runbook adalah dokumen terstandarisasi yang berisi prosedur operasional untuk menanggapi insiden atau melakukan pemulihan (*recovery*) pada sebuah sistem. Dalam ranah operasi TI, *runbook* berfungsi sebagai *playbook* yang mendefinisikan langkah-langkah korektif berurutan berdasarkan jenis insiden, sehingga setiap *operator* yang berbeda mengikuti urutan tindakan yang sama (Schlette et al., 2024). Penggunaan *runbook* mengurangi variabilitas akibat keahlian *operator* yang berbeda-beda dan memastikan bahwa keputusan subjektif diminimalkan selama pemulihan.

Dalam konteks penelitian yang diuraikan dalam proposal ini, *runbook* direpresentasikan sebagai skrip otomasi terstandarisasi (`recovery-runbook.sh`) yang memandu *operator* pada Lingkungan A (manual) melalui prosedur pemulihan yang telah ditentukan sebelumnya (*pre-defined*). Skrip tersebut menampilkan langkah-langkah korektif secara berurutan sesuai jenis penyimpangan, merekam *timestamp* awal dan akhir untuk setiap perintah `kubectl` yang dieksekusi, serta memastikan *operator* tidak melompati langkah atau membuat keputusan di luar prosedur. Representasi *runbook* sebagai skrip otomasi memungkinkan pencatatan waktu yang objektif dan terukur, sekaligus menjaga validitas internal dengan mengontrol variabel pengganggu berupa keahlian *operator*.

2.7 Paradigma *Deployment* Alternatif

Selain paradigma manual dan GitOps yang menjadi fokus penelitian yang diuraikan dalam proposal ini, terdapat paradigma *deployment* lain yang menempati posisi berbeda pada spektrum deklaratif-imperatif. Pertama, paradigma *push-based Continuous Deployment* (misalnya Jenkins CI/CD) mendorong perubahan dari luar kluster ke dalam kluster melalui *pipeline* otomatis. Meskipun menawarkan otomasi *deployment*, paradigma ini memerlukan kredensial kluster yang terekspos pada *pipeline* eksternal dan tidak memiliki mekanisme *reconciliation* bawaan: ketika *state* kluster menyimpang, *pipeline* tidak secara aktif mendeteksi atau mengoreksi deviasi (Utami and Fatta, 2021). Kedua, manajemen konfigurasi imperatif menggunakan alat seperti Ansible atau Terraform menerapkan perubahan melalui perintah imperatif yang dijalankan dari luar kluster. Shrestha and Ali (2024) mengevaluasi Ansible sebagai alternatif GitOps dan menemukan bahwa meskipun Ansible menyediakan otomasi *deployment*, ia tidak memiliki *continuous reconciliation* dan *audit trail* bawaan yang dimiliki GitOps. Ketiga, paradigma perantara — Git sebagai *source of truth* tanpa agen *reconciliation* otomatis — merepresentasikan titik di mana deklarasi *state* disimpan dalam Git, tetapi koreksi penyimpangan tetap bergantung pada intervensi manual.

Perbandingan dalam penelitian yang diuraikan dalam proposal ini difokuskan pada paradigma manual dan GitOps (*pull-based* dengan ArgoCD) karena keduanya merepresentasikan ujung-ujung spektrum yang paling kontras: manual sepenuhnya imperatif tanpa *feedback loop*, dan GitOps sepenuhnya deklaratif dengan *continuous reconciliation*. Perbandingan kedua ujung spektrum ini memungkinkan karakterisasi *magnitude* perbedaan terbesar yang mungkin terjadi, sehingga memberikan batas atas (*upper bound*) bagi estimasi perbedaan antara paradigma-paradigma lain yang menempati posisi antara keduanya.

2.8 Evaluasi Empiris GitOps dan ArgoCD

Utami and Fatta (2021) melakukan analisis perbandingan model *deployment* deklaratif *pull-based* (GitOps dengan ArgoCD) versus *push-based* pada Kubernetes, dan menemukan bahwa model *pull-based* dilaporkan lebih andal karena agen berjalan di dalam kluster dan tidak memerlukan kredensial eksternal. Shrestha and Ali (2024) mengevaluasi GitOps versus Ansible untuk manajemen konfigurasi Kuber-

netes, khususnya pada aspek deteksi penyimpangan dan *remedy time*; namun, studi tersebut mengandalkan penilaian kualitatif untuk menilai tingkat keparahan penyimpangan alih-alih metrik kuantitatif yang dapat direplikasi. Nataraja (2025) melakukan analisis berpusat keamanan terhadap pendekatan deklaratif (GitOps/ArgoCD) dan imperatif (Jenkins/kubectl), menemukan bahwa pendekatan deklaratif menghasilkan koreksi penyimpangan yang lebih cepat, *compliance rate* yang lebih tinggi, dan paparan kerentanan yang lebih rendah; namun, studi tersebut tidak mengisolasi efek pendekatan *deployment* dari variabel pengganggu seperti ukuran klaster dan keahlian tim.

Paavola (2021) membandingkan ArgoCD dan FluxCD untuk *multi-application management* pada Kubernetes dan melaporkan bahwa ArgoCD lebih sesuai untuk skenario multi-application, berkat dukungan UI dan integrasi yang lebih baik. D'Amore (2021) mengimplementasikan ArgoCD untuk *continuous deployment* pada lingkungan *hybrid cloud*, namun tidak membahas evaluasi metrik kuantitatif secara eksplisit. Matubber (2025) mengukur *reconciliation time*, *scalability*, dan *reliability* GitOps untuk infrastruktur K8s, termasuk *self-healing* terhadap penyimpangan konfigurasi; namun, pengukuran dilakukan tanpa *control group* yang menjalankan *manual workflow*, sehingga tidak dapat disimpulkan apakah *reconciliation time* yang terukur berbeda secara bermakna dari pemulihan manual.

Tabel 2.1 merangkum kelemahan metodologis kunci dari penelitian-penelitian tersebut.

Tabel 2.1: Perbandingan Kelemahan Metodologis Penelitian Terdahulu

Penelitian	Fokus	Kelemahan Metodologis
Shrestha and Ali (2024)	GitOps vs Ansible	Penilaian kualitatif untuk tingkat keparahan penyimpangan; tidak ada analisis statistik formal
Nataraja (2025)	Keamanan deklaratif vs imperatif	Tidak mengisolasi variabel pengganggu; desain observasional
Matubber (2025)	<i>Reconciliation time</i> GitOps	Tanpa <i>control group</i> (<i>manual workflow</i>); tidak membandingkan secara terkontrol
Paavola (2021)	ArgoCD vs FluxCD	Komparatif deskriptif; tanpa metrik kuantitatif formal
D'Amore (2021)	ArgoCD <i>hybrid cloud</i>	Studi implementasi; tanpa evaluasi metrik

2.9 Metode Eksperimental Terkontrol pada Riset Sistem

Evaluasi empiris pada riset sistem dan infrastruktur memerlukan desain eksperimental yang memitigasi ancaman terhadap validitas (Wohlin et al., 2012). Limoncelli et al. (2018) menjelaskan bahwa penelitian sistem harus membangun lingkungan yang dapat direproduksi, mengendalikan variabel pengganggu, dan mendefinisikan metrik yang jelas sebelum menyimpulkan hubungan kausal.

Desain eksperimental yang digunakan dalam penelitian yang diuraikan dalam proposal ini membandingkan dua lingkungan yang menerima perlakuan identik (skenario penyimpangan yang sama) dalam kondisi terkontrol: Lingkungan A (*deployment* manual dengan `kubectl`) dan Lingkungan B (*deployment* GitOps dengan ArgoCD). Kedua lingkungan dibangun pada kluster identik dengan konfigurasi yang disamakan; satu-satunya perbedaan adalah paradigma *deployment* (variabel independen), sehingga perbedaan hasil pengukuran dapat diatribusikan kepada perbedaan paradigma (Wohlin et al., 2012).

Analisis data menggunakan uji *Mann–Whitney U* untuk membandingkan setiap metrik antara kedua lingkungan, disertai perhitungan ukuran efek (Cohen’s d atau Cliff’s δ) untuk mengkarakterisasi besarnya perbedaan yang teramati (Wohlin et al., 2012). Prinsip-prinsip rigor yang diterapkan meliputi: (1) pengulangan (*replication*) untuk memastikan reliabilitas; (2) kontrol variabel pengganggu untuk menjaga validitas internal; (3) ukuran efek (*effect size*) untuk menilai signifikansi praktis; dan (4) analisis *power* untuk memastikan ukuran sampel memadai.

2.10 Klasifikasi Beban Uji (*Workload*)

Aplikasi yang dijalankan pada kluster Kubernetes dapat dikelompokkan ke dalam beberapa kelas *workload* berdasarkan karakteristik arsitektur dan manajemen *state* (Siddiqui et al., 2023; Aghili et al., 2024):

1. *Workload* web tanpa state (*stateless web microservice*): Layanan yang tidak menyimpan *state* secara internal, sehingga setiap permintaan dapat dilayani oleh instans mana pun. Contoh: server HTTP sederhana, *API gateway*.
2. *Workload* berbasis data yang menyimpan *state* (*stateful data service*): Layanan yang menyimpan *state* secara persisten, seperti *database* relasional, *search engine*, atau *object storage*. *Workload* kelas ini memerlukan urutan *deployment* dan penanganan penyimpangan yang lebih kompleks.

3. *Workload* komposit multi-service: Aplikasi yang terdiri atas banyak komponen yang saling bergantung, mencakup kombinasi kelas 1 dan 2 beserta komponen infrastruktur (*ingress*, *TLS*, *secrets*, *monitoring*, *backup*, kebijakan keamanan). Kelas *workload* ini memiliki kompleksitas dependensi tertinggi dan paling rentan terhadap penyimpangan konfigurasi akibat banyaknya komponen yang harus diselaraskan.

Penelitian yang diuraikan dalam proposal ini menggunakan beban uji (*workload*) komposit multi-service yang mencakup setidaknya 16 komponen: (1) *web application*, (2) *Celery workers* untuk pemrosesan asinkron, (3) *database* relasional (PostgreSQL), (4) *search engine* (OpenSearch), (5) *cache* dan *message broker* (Redis), (6) *object storage* (MinIO/S3), serta komponen infrastruktur pendukung meliputi (7) *ingress controller*, (8) *certificate manager*, (9) *secret encryption*, (10) *network tunnel*, (11) kebijakan keamanan, (12) *monitoring system*, dan (13) *backup*. Beban uji dipilih karena memenuhi tiga kriteria: (a) kompleksitas dependensi yang memerlukan urutan *deployment*, di mana layanan *web* bergantung pada *database*, *cache*, dan *search engine*; (b) keberadaan komponen infrastruktur yang luas, mencakup *ingress*, *TLS*, *secrets*, *monitoring*, *backup*, dan kebijakan keamanan; dan (c) ketersediaan sebagai platform *open-source* yang dapat direproduksi. Dalam konteks penelitian yang diuraikan dalam proposal ini, beban uji berfungsi sebagai subjek uji (*test subject*), bukan objek penelitian. Segala temuan dan metrik berlaku untuk kelas *workload* komposit multi-service secara umum.

2.11 Kesenjangan Penelitian (*Research Gap*)

Berdasarkan tinjauan pustaka yang telah diuraikan, terdapat empat kesenjangan yang saling berkaitan dan menjadi landasan penelitian:

1. Kesenjangan Validitas Konstruk: Penelitian sebelumnya menunjukkan bahwa GitOps mengurangi penyimpangan konfigurasi, namun temuan-temuan tersebut bergantung pada metrik yang tidak teroperasionalisasi secara kuantitatif. Shrestha and Ali (2024) menggunakan penilaian kualitatif untuk tingkat keparahan penyimpangan, sementara Nataraja (2025) mengukur *compliance rate* tanpa mendefinisikan protokol pengukuran yang dapat direplikasi. Tanpa operasionalisasi metrik yang tegas, klaim karakteristik GitOps tidak dapat diverifikasi secara independen.

2. Kesenjangan Validitas Internal: Studi-studi yang ada tidak mengisolasi efek paradigma *deployment* dari variabel pengganggu. Nataraja (2025) tidak mengendalikan ukuran klaster atau keahlian *operator*; Matubber (2025) mengukur *reconciliation time* GitOps tanpa *control group* (*manual workflow*). Akibatnya, efek yang diamati tidak dapat diatribusikan secara kausal kepada GitOps, yang mungkin disebabkan oleh keahlian *operator* yang lebih tinggi, karakteristik *workload* yang lebih sederhana, atau kondisi klaster yang lebih stabil.
3. Kesenjangan Metodologis: Tidak ditemukan penelitian yang menerapkan desain eksperimental terkontrol dengan analisis statistik formal (uji komparatif dan ukuran efek) pada perbandingan GitOps versus manual. Absensi rigor statistik tersebut mengakibatkan temuan yang ada tidak dapat digeneralisasi dan rentan terhadap *Type I error* (menerima klaim positif yang sebenarnya palsu).
4. Kesenjangan Kontekstual: Evaluasi GitOps yang ada dilakukan pada konteks yang bervariasi tanpa standarisasi metrik dan protokol, sehingga temuan tidak dapat dibandingkan lintas studi. Belum ada evaluasi yang menggunakan desain eksperimental terkontrol pada konteks klaster yang mengelola *workload* komposit multi-service dengan dependensi lintas komponen, di mana satu insiden penyimpangan dapat memengaruhi ketersediaan layanan secara keseluruhan.

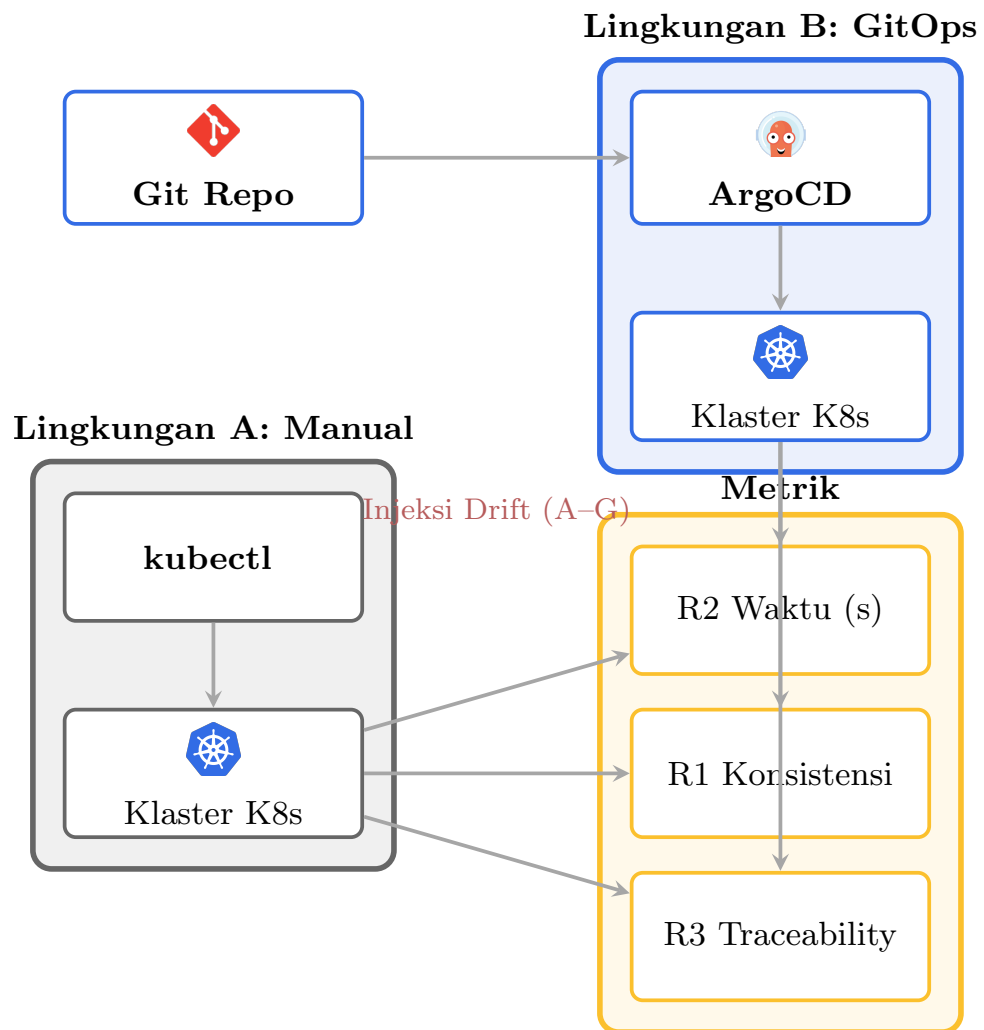
Penelitian yang diuraikan dalam proposal ini mengisi keempat kesenjangan tersebut dengan: (1) mengoperasionalisasi tiga metrik kuantitatif yang dapat direplikasi; (2) mengendalikan variabel pengganggu melalui desain eksperimental terkontrol; (3) menerapkan analisis statistik komparatif dengan uji signifikansi dan ukuran efek; dan (4) mengevaluasi pada konteks *workload* komposit multi-service dengan kompleksitas dependensi yang representatif.

BAB III

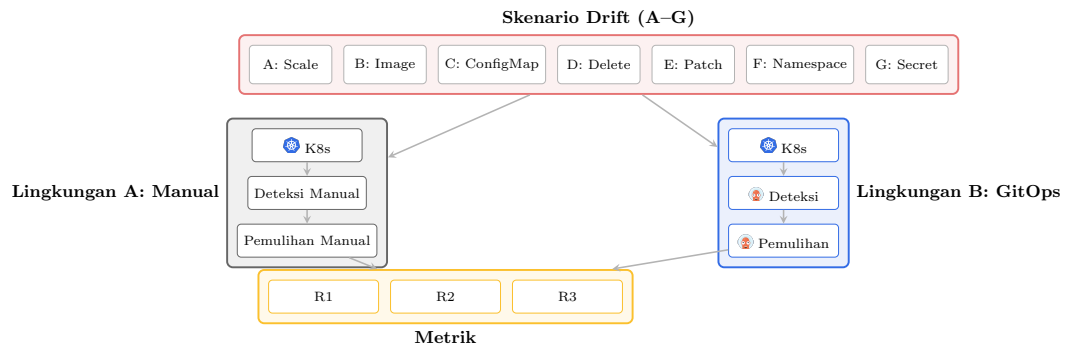
METODE DAN RANCANGAN PENELITIAN

3.1 Deskripsi Umum Penelitian

Penelitian yang diuraikan dalam proposal ini mengevaluasi secara empiris dampak skenario penyimpangan konfigurasi terhadap karakteristik operasional *deployment* pada dua paradigma manajemen yang berbeda, yaitu manual dan GitOps, pada sebuah kluster Kubernetes. Evaluasi dilakukan dengan membandingkan dua lingkungan paralel: Lingkungan A (*deployment* manual menggunakan `kubectl`) dan Lingkungan B (*deployment* GitOps menggunakan ArgoCD). Pada kedua lingkungan, penyimpangan konfigurasi di-*inject* secara terkontrol melalui tujuh skenario yang dirancang untuk merepresentasikan kesalahan operasional yang umum terjadi. Hasil pengukuran terhadap tiga metrik utama (konsistensi *deployment*, waktu deteksi/pemulihan/identifikasi akar masalah, dan *traceability*) dianalisis menggunakan uji statistik komparatif untuk mengkarakterisasi perbedaan antara kedua paradigma. Hasil evaluasi tersebut mengkarakterisasi besarnya dan kondisi perbedaan antara kedua paradigma, yang selanjutnya menjadi dasar rekomendasi pemilihan paradigma *deployment* bagi praktisi industri.



Gambar 3.1: Arsitektur eksperimen: perbandingan Lingkungan A (*deployment* manual dengan `kubectl`) dan Lingkungan B (*deployment* GitOps dengan ArgoCD). Injeksi penyimpangan dilakukan pada kedua lingkungan, dan metrik R1–R3 dikumpulkan dari masing-masing lingkungan.



Gambar 3.2: Skenario penyimpangan terkontrol (A–G): alur injeksi pada kedua lingkungan, mekanisme deteksi dan pemulihan pada Lingkungan A (manual) dan Lingkungan B (GitOps/ArgoCD), serta tiga metrik yang dikumpulkan.

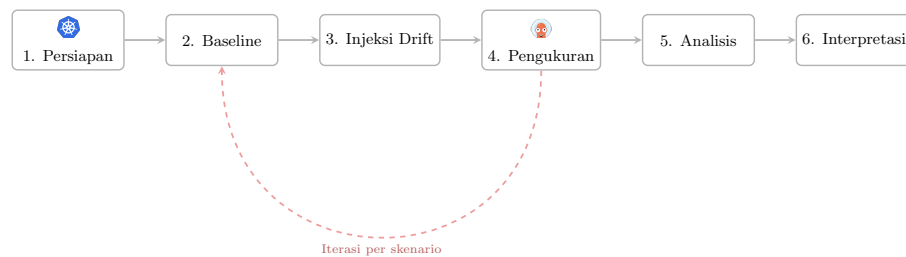
3.2 Metodologi

Metode penelitian yang digunakan adalah Evaluasi Empiris Terkontrol. Dalam desain ini, dua lingkungan paralel dibangun pada kluster Kubernetes yang identik: Lingkungan A (*deployment* manual) dan Lingkungan B (*deployment* GitOps). Kedua lingkungan menerima perlakuan identik berupa injeksi penyimpangan melalui skenario yang sama, sehingga satu-satunya variabel yang berbeda adalah paradigma *deployment*. Perbedaan hasil pengukuran antara kedua lingkungan dapat diatribusikan kepada perbedaan paradigma *deployment*. Urutan skenario diacak untuk menghindari efek urutan (*order effect*), dan variabel pengganggu dikendalikan melalui konfigurasi kluster yang disamakan (Wohlin et al., 2012).

Tahapan eksperimen mengikuti rancangan *pre-test – treatment – post-test*:

1. Tahap 1: Persiapan Lingkungan. Pembangunan Lingkungan A (manual) dan Lingkungan B (GitOps) pada kluster Kubernetes yang identik. Konfigurasi kluster, versi ArgoCD, dan spesifikasi *node* disamakan untuk mengendalikan variabel pengganggu.
2. Tahap 2: Kalibrasi *Baseline* (O1). Pengukuran *baseline* pada keadaan bersih sebelum injeksi penyimpangan. Mencakup verifikasi status *deployment*, pengukuran awal konsistensi, dan pencatatan metrik lingkungan (penggunaan CPU/memori, latensi jaringan).

3. Tahap 3: Injeksi Penyimpangan (X). Injeksi penyimpangan konfigurasi terkontrol pada kedua lingkungan menggunakan perintah identik (mis. `kubectl edit`, `kubectl scale`, `kubectl patch`). Urutan skenario diacak untuk menghindari efek urutan (*order effect*).
4. Tahap 4: Pengukuran Pasca-Perlakuan (O2). Pengumpulan data metrik pasca-injeksi, termasuk waktu deteksi, waktu pemulihan, waktu identifikasi akar masalah, dan dokumentasi *traceability*.
5. Tahap 5: Analisis Statistik Komparatif. Penerapan uji *Mann–Whitney U* serta perhitungan ukuran efek (Cohen’s d atau Cliff’s δ) untuk mengkarakterisasi perbedaan antara kedua paradigma.
6. Tahap 6: Interpretasi dan Kesimpulan. Pembahasan hasil statistik dalam konteks literatur dan praktik operasional, serta perumusan rekomendasi paradigma.



Gambar 3.3: Bagan alir eksperimen: tahapan penelitian dari persiapan hingga interpretasi. Iterasi per skenario menunjukkan pengulangan acak dari *baseline* hingga pengukuran untuk setiap skenario penyimpangan.

3.3 Definisi Intervensi dan Variabel Penelitian

3.3.1 Intervensi (Perlakuan)

Intervensi dalam penelitian yang diuraikan dalam proposal ini adalah injeksi penyimpangan terkontrol melalui tujuh skenario (A–G). Intervensi bersifat identik pada kedua lingkungan, yakni perintah `kubectl` yang sama digunakan untuk menginjeksi penyimpangan pada Lingkungan A dan Lingkungan B. Yang membedakan kedua lingkungan bukanlah perlakuannya, melainkan paradigma manajemen *deployment* yang menentukan bagaimana sistem merespons terhadap penyimpangan yang terjadi.

3.3.2 Variabel Independen (Variabel Bebas)

Variabel independen adalah paradigma manajemen *deployment* dengan dua level:

1. Lingkungan A, *Deployment Manual*: *Deployment* dan modifikasi dilakukan langsung pada klaster menggunakan `kubectl apply`, `kubectl edit`, `kubectl scale`, dan perintah imperatif lainnya tanpa repositori Git sebagai *source of truth*.
2. Lingkungan B, *Deployment GitOps (ArgoCD)*: *Deployment* dikelola secara deklaratif melalui repositori Git; ArgoCD berjalan di dalam klaster sebagai agen *reconciliation* yang secara kontinu menyelaraskan *state* klaster dengan *state* di Git.

3.3.3 Variabel Dependen (Variabel Terikat)

Tiga metrik utama diukur sebagai variabel dependen:

Metrik R1: Konsistensi Deployment Definisi: Persentase sumber daya klaster (*Pod*, *Deployment*, *ConfigMap*, *Service*) yang cocok dengan *desired declarative state* setelah skenario penyimpangan.

Cara Ukur:

$$\text{Konsistensi (\%)} = \frac{\text{Jumlah sumber daya dengan status Healthy/Synced}}{\text{Total sumber daya yang diharapkan}} \times 100$$

Untuk Lingkungan B, status diambil dari ArgoCD *Application status*. Untuk Lingkungan A, status diverifikasi menggunakan `kubectl get` dan `kubectl describe`. Konsistensi diukur pada t_0 (sebelum penyimpangan), t_1 (sebelum pemulihan), dan t_2 (setelah pemulihan).

Metrik R2: Waktu Deteksi, Pemulihan, dan Identifikasi Akar Masalah Definisi: Tiga sub-metrik waktu (dalam detik) yang mengukur kecepatan respons terhadap penyimpangan konfigurasi.

- Waktu Deteksi (*Detection Time*, $t_d - t_0$): Waktu dari injeksi penyimpangan hingga penyimpangan terdeteksi. Untuk Lingkungan B, ArgoCD menunjukkan

status *OutOfSync* pada aplikasi yang terkena penyimpangan (dipantau menggunakan `argocd app get` dan log ArgoCD). Untuk Lingkungan A, *operator* mendeteksi deviasi secara manual melalui `kubectl get` atau *monitoring*.

- Waktu Pemulihan (*Recovery Time*, $t_2 - t_0$): Waktu dari injeksi penyimpangan hingga *state* klaster kembali ke *baseline*. Untuk Lingkungan B, ArgoCD menyelesaikan *automatic reconciliation (self-heal)* dan menunjukkan status *Synced/Healthy*. Untuk Lingkungan A, *operator* menyelesaikan seluruh perintah `kubectl` manual yang diperlukan untuk mengembalikan *state* ke *baseline*.
- Waktu Identifikasi Akar Masalah (*Root Cause Identification Time*, $t_{rci} - t_d$): Waktu dari deteksi penyimpangan hingga penyebab penyimpangan teridentifikasi. Untuk Lingkungan B, ArgoCD menampilkan *diff* antara *state* Git dan *state* klaster, sehingga akar masalah dapat langsung diidentifikasi. Untuk Lingkungan A, *operator* harus memeriksa log, menjalankan `kubectl describe`, dan menganalisis perubahan secara manual untuk menentukan penyebab penyimpangan.

Pengamatan Tambahan: Jumlah perintah `kubectl` yang diperlukan untuk pemulihan pada Lingkungan A dicatat sebagai pengamatan pelengkap, bukan sebagai metrik independen, karena waktu pemulihan (*recovery time*) secara inheren sudah mencerminkan kompleksitas pemulihan.

Pengulangan: Setiap skenario diulang $n = 10$ kali per lingkungan untuk menghasilkan distribusi waktu yang memadai untuk analisis statistik.

Metrik R3: *Traceability* Operasional (Deskriptif) Definisi: Kemampuan untuk merekonstruksi siapa yang mengubah *state*, apa yang berubah, kapan perubahan terjadi, dan bagaimana pemulihan dilakukan.

Cara Ukur: Metrik ini dinilai secara deskriptif, bukan diuji secara statistik. Untuk setiap skenario penyimpangan, aspek *traceability* berikut didokumentasikan untuk kedua lingkungan:

- Lingkungan B (GitOps): Setiap perubahan direkam secara otomatis melalui Git *commit history* (penulis, *timestamp*, *diff*) dan ArgoCD *notifications*. Kelengkapan *audit log* dihitung sebagai persentase perubahan yang dapat direkonstruksi dari Git *commit history*.

- Lingkungan A (Manual): *Traceability* bergantung pada Kubernetes *audit logs* dan pencatatan manual. Kelengkapan *audit log* dihitung sebagai persentase perubahan yang dapat direkonstruksi dari *audit logs*.

Penjelasan Metodologis: Metrik *traceability* tidak diuji secara statistik karena perbedaan struktural antara kedua paradigma sudah terjamin oleh desain sistem: Git secara inheren menyediakan *audit trail* lengkap (*commit hash*, penulis, *timestamp*, *diff*), sementara *manual deployment* bergantung pada *audit logs* yang parsial. Perbandingan kuantitatif pada dimensi ini bersifat *predetermined*. Sebagai gantinya, *traceability* dideskripsikan secara kualitatif dalam pembahasan hasil, mencakup: kelengkapan jejak perubahan, kemudahan identifikasi akar masalah, dan keterkaitan dengan waktu identifikasi akar masalah (R2).

3.4 Justifikasi Ekuitas Perbandingan (*Apple-to-Apple*)

Untuk memastikan bahwa perbandingan antara paradigma manual dan GitOps bersifat setara (*apple-to-apple*), dirumuskan kerangka argumentatif berikut yang menjelaskan bagaimana variabel dikendalikan sehingga perbedaan hasil pengukuran dapat diatribusikan secara kausal kepada perbedaan paradigma *deployment* (Wohlin et al., 2012):

1. Identitas Klaster: Kedua lingkungan dibangun pada klaster Kubernetes yang identik dengan spesifikasi *hardware*, versi Kubernetes, *container runtime*, dan sistem operasi *node* yang sama. Hal ini mengeliminasi perbedaan akibat infrastruktur fisik.
2. Identitas *Workload*: Kedua lingkungan menjalankan *deployment* yang sama (beban uji kelas *workload* komposit multi-service) dengan konfigurasi manifest yang identik. Hal ini mengeliminasi perbedaan akibat kompleksitas atau jenis aplikasi.
3. Identitas Intervensi: Injeksi penyimpangan dilakukan menggunakan perintah `kubectl` yang sama pada kedua lingkungan, sehingga jenis dan magnitudo penyimpangan identik. Yang membedakan adalah bagaimana masing-masing paradigma merespons, bukan apa yang diinjeksi.

4. Standarisasi *Operator*: Pada Lingkungan A, *operator* mengikuti *recovery runbook* terstandarisasi yang mengeliminasi keputusan subjektif. Seluruh *timestamp* direkam oleh skrip otomasi, bukan oleh penilaian manusia. Hal ini mengontrol variabel pengganggu berupa keahlian *operator* (dibahas lebih lanjut pada § 3.5.1).
5. Kontrol Lingkungan: Variabel pengganggu seperti tekanan sumber daya *node*, latensi jaringan, dan interval *sync* ArgoCD dikendalikan dan dipantau secara kontinu (dibahas pada § 3.7.5).
6. Isolasi Variabel Independen: Satu-satunya perbedaan antara kedua lingkungan adalah paradigma manajemen *deployment* (variabel independen). Semua variabel lain disamakan, sehingga perbedaan hasil pengukuran dapat diatribusikan kepada perbedaan paradigma.

Kerangka tersebut memenuhi kriteria desain eksperimental terkontrol sebagaimana dijelaskan oleh Wohlin et al. (2012), di mana variabel independen dimanipulasi sementara variabel pengganggu dikendalikan.

3.5 Peran Peneliti dan Standarisasi Prosedur

Dalam penelitian yang diuraikan dalam proposal ini, peneliti berperan sebagai instrumen pengukuran yang terstandarisasi, bukan sebagai variabel subjek. Hal tersebut penting untuk menjaga validitas internal mengingat peneliti juga bertindak sebagai *operator* pada Lingkungan A. Peran dan batasan peneliti didefinisikan sebagai berikut:

1. Pada Lingkungan A (Manual): Peneliti mengeksekusi prosedur pemulihan berdasarkan *recovery runbook* yang telah ditentukan sebelumnya (*pre-defined*). *Operator* tidak membuat keputusan subjektif selama pemulihan, sehingga setiap langkah korektif mengikuti skrip yang telah ditulis. Seluruh *timestamp* (awal dan akhir setiap perintah) direkam secara otomatis oleh skrip otomasi, bukan oleh pencatatan manual.
2. Pada Lingkungan B (GitOps): ArgoCD beroperasi secara otomatis; peneliti hanya memantau dan merekam data. Tidak ada keputusan operasional yang dibuat oleh peneliti pada lingkungan tersebut.

3. Pencatatan data: Seluruh pengukuran waktu (R2) dan konsistensi (R1) direkam oleh skrip otomasi, bukan oleh penilaian manusia. *Traceability* (R3) didokumentasikan berdasarkan bukti objektif (log dan Git *history*) tanpa penilaian subjektif.

Batasan penggunaan *operator* tunggal diakui secara eksplisit sebagai ancaman terhadap validitas eksternal (dibahas pada § 3.14) dan direkomendasikan untuk replikasi dengan tim *operator* yang berbeda.

3.5.1 Mitigasi Bias Keahlian *Operator* pada Waktu Identifikasi Akar Masalah

Waktu identifikasi akar masalah (*root cause identification time*, R2) pada Lingkungan A bergantung pada keahlian *operator* dalam memeriksa log dan menganalisis perubahan. Perbedaan keahlian *operator* dapat memengaruhi waktu yang terukur, yang berpotensi menjadi bias. Metodologi berikut diterapkan untuk memitigasi bias tersebut:

1. *Recovery runbook* terstandarisasi: Setiap langkah identifikasi akar masalah di-*encode* dalam *runbook* sebagai urutan pemeriksaan yang deterministik (mis. periksa `kubectl describe`, lalu `kubectl logs`, lalu `kubectl get events`), sehingga keahlian subjektif diminimalkan (Schlette et al., 2024).
2. *Practice run* sebelum pengumpulan data: *Operator* melakukan pengulangan latihan hingga mencapai *plateau* waktu identifikasi, memastikan efek pembelajaran (*maturation*) telah stabil sebelum data resmi dikumpulkan.
3. Profil keahlian *operator*: Sebelum eksperimen, profil keahlian *operator* didokumentasikan secara transparan, mencakup: (a) pengalaman Kubernetes dalam tahun; (b) sertifikasi yang dimiliki (CKA, CKAD, atau setara); (c) frekuensi penggunaan `kubectl` dalam praktik sehari-hari; dan (d) pengalaman sebelumnya dengan *incident response*. Profil ini berfungsi untuk kontekstualisasi hasil, bukan sebagai variabel independen.
4. Survei beban kognitif NASA-TLX: Setelah setiap skenario pada Lingkungan A, *operator* mengisi kuesioner NASA-TLX (*Task Load Index*) (Hart and Staveland, 1988) untuk mengukur beban kognitif yang dirasakan selama proses

pemulihan. Survei ini mencakup enam dimensi: *mental demand*, *physical demand*, *temporal demand*, *performance*, *effort*, dan *frustration*. Data NASA-TLX digunakan sebagai *measured confound* untuk mengkontekstualisasi hasil R2, bukan sebagai metrik independen.

5. Survei kesulitan pasca-eksperimen: Pada akhir eksperimen, *operator* menilai tingkat kesulitan setiap skenario pada skala Likert 1–5, memberikan data persepsional yang melengkapi pengukuran waktu objektif (R2).
6. Rekomendasi replikasi multi-*operator*: Batasan *operator* tunggal diakui secara eksplisit, dan replikasi dengan *operator* yang berbeda tingkat keahlian direkomendasikan sebagai penelitian lanjutan untuk mengvalidasi generalisasi temuan.

Pada Lingkungan B, waktu identifikasi akar masalah tidak bergantung pada keahlian *operator* karena ArgoCD menampilkan *diff* antara *state* Git dan *state* kluster secara otomatis, sehingga akar masalah dapat langsung diidentifikasi tanpa analisis manual. Perbedaan struktural ini merupakan salah satu karakteristik paradigma yang dievaluasi, bukan bias yang perlu dieliminasi.

3.6 Skrip Otomasi Eksperimen

Untuk memastikan konsistensi prosedural dan mengurangi bias eksperimenter, empat kategori skrip otomasi dikembangkan:

3.6.1 Skrip Injeksi Penyimpangan (`drift-inject.sh`)

Skrip ini mengotomasi injeksi skenario penyimpangan pada kedua lingkungan:

- Memilih skenario secara acak dari himpunan {A, B, C, D, E, F, G}.
- Mengeksekusi perintah `kubectl` yang sesuai pada kedua lingkungan secara paralel.
- Merekam *timestamp* t_0 (waktu injeksi) ke log terstruktur (JSON/CSV).
- Memverifikasi bahwa penyimpangan berhasil diterapkan sebelum melanjutkan ke tahap pengukuran.

3.6.2 Skrip Pengumpulan Metrik (`metrics-collect.sh`)

Skrip ini melakukan *polling* berkala terhadap kedua lingkungan:

- Lingkungan B: Memanggil ArgoCD API (`argocd app get`) setiap 5 detik untuk mendeteksi perubahan status (*Synced* $\rightarrow$ *OutOfSync* $\rightarrow$ *Synced*).
- Lingkungan A: Menjalankan `kubectl get` dan `kubectl describe` secara berkala untuk memantau status sumber daya.
- Merekam semua *timestamp* secara otomatis (waktu deteksi t_d , waktu pemulihan t_2).
- Menghitung skor konsistensi (R1) secara otomatis berdasarkan perbandingan *state* aktual dengan *desired state*.

3.6.3 Skrip Verifikasi *Baseline* (`baseline-check.sh`)

Skrip ini memverifikasi kondisi klaster sebelum setiap *run* eksperimental:

- Memeriksa bahwa semua *pod* dalam status *Running* dan sehat.
- Memastikan penggunaan CPU/memori *node* di bawah 70% kapasitas.
- Memverifikasi bahwa ArgoCD menunjukkan status *Synced/Healthy* pada Lingkungan B.
- Menolak memulai eksperimen jika syarat *baseline* tidak terpenuhi; mencatat pelanggaran ke log.

3.6.4 Skrip *Recovery Runbook* (`recovery-runbook.sh`)

Skrip ini memandu *operator* pada Lingkungan A melalui prosedur pemulihan yang terstandarisasi:

- Menampilkan langkah-langkah korektif secara berurutan sesuai jenis penyimpangan yang terjadi.
- Merekam *timestamp* awal dan akhir untuk setiap perintah `kubectl` yang dieksekusi.

- Menghitung jumlah perintah `kubectl` yang dieksekusi sebagai pengamatan pelengkap.
- Memastikan *operator* tidak melompati langkah atau membuat keputusan di luar prosedur.

Seluruh skrip akan dipublikasikan sebagai *artifact* pendamping untuk mendukung reproduisibilitas penelitian.

3.7 Lingkungan Eksperimen

3.7.1 Klaster Kubernetes

Penelitian menggunakan klaster Kubernetes yang dikelola Rancher dengan spesifikasi *hardware* berikut:

- Jumlah *Node*: 3 *node*
- Total CPU: 24 inti
- Total Memori: ~23 Gi
- Versi Kubernetes: v1.29+
- *Container Runtime*: containerd
- Sistem Operasi *Node*: Ubuntu 22.04 LTS

3.7.2 ArgoCD (Lingkungan B)

- Versi ArgoCD: v2.12+
- Konfigurasi *Sync*: *Auto-sync* diaktifkan. Interval *sync* ditetapkan pada 60 detik untuk konsistensi waktu deteksi.
- Konfigurasi *Self-Heal*: Diaktifkan untuk seluruh eksperimen, sehingga ArgoCD secara otomatis mendeteksi dan mengoreksi penyimpangan.
- Konfigurasi *Prune*: *Pruning* diaktifkan agar sumber daya yang dihapus secara manual (Skenario D) dapat dideteksi dan diciptakan kembali oleh ArgoCD.

3.7.3 Beban Uji (*Workload*)

Workload yang digunakan adalah kelas *workload* komposit multi-service, yang mencakup setidaknya 16 komponen infrastruktur dan dependensi, termasuk *web application*, *worker* asinkron, *database* relasional, *search engine*, *cache*, *object storage*, *ingress controller*, *certificate manager*, *secret encryption*, *monitoring system*, *backup*, dan kebijakan keamanan. Beban uji berfungsi sebagai subjek uji, bukan objek penelitian; temuan berlaku untuk kelas *workload* komposit multi-service secara umum.

3.7.4 Konfigurasi dan Dataset

Eksperimen melibatkan dua kategori artefak yang perlu dibedakan secara eksplisit: konfigurasi dan dataset.

Konfigurasi adalah artefak yang mendefinisikan *desired state* dan tata kelola lingkungan eksperimen, meliputi: (1) Manifest Kubernetes deklaratif untuk seluruh komponen beban uji (Helm *chart* atau Kustomize *manifests*), disimpan dalam repositori Git sebagai *source of truth* untuk Lingkungan B; (2) konfigurasi *baseline* yang disamakan untuk kedua lingkungan, mencakup jumlah replika, versi *image*, isi ConfigMap, dan Secret yang identik pada awal setiap *run*; serta (3) konfigurasi ArgoCD (*sync policy*, *self-heal*, *prune*) yang menentukan perilaku *reconciliation* pada Lingkungan B.

Dataset adalah data empiris yang dikumpulkan selama eksperimen, meliputi: (1) log terstruktur (JSON/CSV) yang merekam *timestamp* t_0 , t_d , t_{rci} , dan t_2 untuk setiap skenario dan pengulangan; (2) snapshot status klaster (`kubectl get all`) sebelum dan setelah setiap skenario sebagai bukti konsistensi (R1); serta (3) data *traceability*, yaitu Git *commit history* pada Lingkungan B dan Kubernetes *audit logs* pada Lingkungan A, sebagai sumber bukti untuk metrik R3.

3.7.5 Kontrol Variabel Pengganggu

Agar validitas internal tetap tinggi, variabel berikut dikendalikan secara sistematis:

1. Interval Sync ArgoCD: Ditetapkan pada 60 detik (bukan *default* 180 detik) agar waktu deteksi konsisten.

2. Tekanan Sumber Daya *Node*: Merekam penggunaan CPU/memori kluster sebelum setiap pengujian. Eksperimen hanya dijalankan jika *node* berada di bawah 70% kapasitas.
3. Latensi Jaringan: Menggunakan kluster *on-premise* dengan latensi stabil. RTT antar-*node* dicatat.
4. Waktu Injeksi Penyimpangan: Injeksi selalu dilakukan pada interval *sync* ArgoCD untuk menghindari *edge case* waktu tunggu yang tidak dapat diprediksi.
5. Versi ArgoCD: Menggunakan satu versi ArgoCD (v2.12+) sepanjang eksperimen.

3.7.6 Protokol *Monitoring Variabel Terkontrol*

Variabel pengganggu dipantau secara kontinu selama eksperimen menggunakan `baseline-check.sh` yang mencatat metrik sistem setiap 10 detik. Jika selama eksperimen terjadi pelanggaran batas (misalnya penggunaan CPU melebihi 70%), eksperimen dihentikan, data dari *run* tersebut dibuang, lingkungan dikalibrasi ulang ke *baseline*, dan *run* diulang. Seluruh kejadian pelanggaran dicatat dalam log eksperimen untuk transparansi.

3.8 Analisis Power dan Justifikasi Ukuran Sampel

Ukuran sampel ditentukan berdasarkan analisis *power a priori* menggunakan pendekatan berikut:

1. Ukuran efek target: Berdasarkan temuan Nataraja (2025) yang melaporkan perbedaan substansial antara pendekatan deklaratif dan imperatif, ukuran efek besar ($d = 1,0$, konvensi Cohen) digunakan sebagai estimasi konservatif.
2. *Power* dan signifikansi: *Statistical power* ditetapkan pada $1 - \beta = 0,80$ dan $\alpha = 0,05$ (*two-tailed*).
3. Ukuran sampel minimum (per grup): Berdasarkan konvensi Cohen untuk ukuran efek besar ($d = 1,0$), $\alpha = 0,05$, dan *power* = 0,80, ukuran sampel minimum per grup adalah $n = 17$ secara total per metrik (Cohen, 1988). Namun, karena setiap skenario diulang secara independen (7 skenario $\times$ n pengulangan per skenario), jumlah total observasi per metrik per lingkungan adalah $7 \times n$.

4. Pemilihan n per skenario: Untuk R2 (waktu deteksi, waktu pemulihan, dan waktu identifikasi akar masalah, metrik utama), dipilih $n = 10$ per skenario per lingkungan, menghasilkan $7 \times 10 = 70$ observasi per lingkungan, jauh melampaui minimum $n = 17$ secara total. Untuk R1 (konsistensi), dipilih $n = 5$ per skenario per lingkungan, menghasilkan $7 \times 5 = 35$ observasi per lingkungan, masih memadai untuk mendeteksi efek besar. R3 (*traceability*) bersifat deskriptif dan tidak memerlukan ukuran sampel statistik. Jika hasil menunjukkan efek sedang ($d = 0,5$), pengulangan dapat ditambah pada tahap pengumpulan data tambahan.

3.9 Matriks Eksperimen

Tabel 3.1 menunjukkan rancangan eksperimental secara keseluruhan.

Tabel 3.1: Matriks Eksperimen

Dimensi	Lingkungan A (Manual)	Lingkungan B (GitOps)
Paradigma	kubectl imperatif	ArgoCD deklaratif + Git
Intervensi	Skenario penyimpangan A-G (identik)	Skenario penyimpangan A-G (identik)
Pengulangan	$n = 5$ per skenario (R1); $n = 10$ per skenario (R2); R3 deskriptif	$n = 5$ per skenario (R1); $n = 10$ per skenario (R2); R3 deskriptif
Metrik	R1, R2, R3	R1, R2, R3
Baseline	kubectl get semua sumber daya sebelum penyimpangan	ArgoCD App status sebelum penyimpangan

3.10 Skenario Penyimpangan Terkontrol

Tujuh skenario penyimpangan dirancang untuk merepresentasikan kesalahan operasional yang umum. Pemilihan skenario didasarkan pada taksonomi *defect* konfigurasi Kubernetes oleh Zhang et al. (2026) dan praktik operasional yang dilaporkan oleh Pohjola (2025), yang mengidentifikasi kategori kesalahan operasional berulang pada platform berbasis konfigurasi deklaratif. Setiap skenario dieksekusi secara identik pada kedua lingkungan. Untuk Lingkungan A, *operator* kemudian memulihkan secara manual mengikuti *recovery runbook*. Untuk Lingkungan B, ArgoCD secara otomatis mendeteksi dan memulihkan penyimpangan (*self-heal* aktif).

3.10.1 Skenario A: Perubahan Replika (*Replica Drift*)

Secara manual mengubah jumlah replika *Deployment* menggunakan `kubectl scale deployment/nama-deployment --replicas=5` ketika *desired* adalah 3. Perubahan tersebut melewati Git pada Lingkungan B.

3.10.2 Skenario B: Perubahan *Image* (*Image Drift*)

Secara manual mengganti versi *image* kontainer pada *Deployment* menggunakan `kubectl edit deployment/nama-deployment` dan mengubah kolom *image* ke versi yang berbeda.

3.10.3 Skenario C: Perubahan ConfigMap (*ConfigMap Drift*)

Mengubah isi ConfigMap langsung di dalam kluster menggunakan `kubectl edit configmap/nama-configmap`. Memodifikasi parameter konfigurasi seperti `DATABASE_URL` atau `LOG_LEVEL`.

3.10.4 Skenario D: Penghapusan Sumber Daya (*Resource Deletion*)

Menghapus *Deployment* atau *Service* secara manual menggunakan `kubectl delete deployment/nama-deployment`. Untuk Lingkungan B, ArgoCD dengan *prune* aktif akan mendeteksi sumber daya yang hilang dan menciptakan ulang dari manifest Git.

3.10.5 Skenario E: *Patch* Manual Tidak Sah (*Unauthorized Manual Patch*)

Menerapkan `kubectl patch` langsung terhadap sumber daya yang dikelola, mis. mengubah *resource limits* pada sebuah *Pod* menggunakan *patch* JSON.

3.10.6 Skenario F: Penyimpangan Lintas *Namespace* (*Cross-namespace Drift*)

Menerapkan sumber daya ke *namespace* yang salah secara manual. Mengukur apakah GitOps membatasi propagasi atau mendeteksi kesalahan *namespace* secara otomatis. Contoh: `kubectl apply -f deployment.yaml --namespace=wrong-namespace`.

3.10.7 Skenario G: Perubahan Secret (*Secret Drift*)

Mengubah *secret* secara manual di luar Git. Skenario tersebut umum dalam praktik produksi. Contoh perintah:

Listing III.1: Contoh perubahan Secret

```
1 kubectl patch secret my-secret \
2   --patch='{"data":{"password":"<new_base64>"}}'
```

3.11 Protokol Pengukuran

3.11.1 Protokol Pengukuran *Baseline* (O1)

Sebelum setiap *run*:

1. Jalankan `baseline-check.sh` untuk memverifikasi kondisi klaster.
2. Verifikasi semua *pod* dalam status *Running* dan sehat.
3. Catat *timestamp* awal.
4. Simpan output `kubectl get all -n <namespace>` (Lingkungan A) atau `argocd app list` (Lingkungan B) sebagai *baseline*.
5. Verifikasi variabel pengganggu berada dalam batas (CPU < 70%, RTT stabil).

3.11.2 Protokol Injeksi Penyimpangan (X)

1. Jalankan `drift-inject.sh` yang memilih skenario secara acak dari daftar {A, B, C, D, E, F, G}.
2. Skrip mengeksekusi perintah injeksi penyimpangan pada kedua lingkungan secara paralel.
3. Skrip mencatat *timestamp* t_0 (waktu eksekusi perintah injeksi) secara otomatis.

3.11.3 Protokol Pengukuran Pasca-Injeksi (O2)

1. Untuk Lingkungan B (GitOps): `metrics-collect.sh` memantau status ArgoCD setiap 5 detik. Catat: (a) waktu ArgoCD menunjukkan *OutOfSync*

(deteksi); (b) waktu ArgoCD menunjukkan *Synced* setelah *auto-heal* (pemulihan); (c) waktu yang diperlukan untuk mengidentifikasi akar masalah dari *diff* yang ditampilkan ArgoCD (identifikasi akar masalah). Hitung $t_d - t_0$ (waktu deteksi), $t_2 - t_0$ (total waktu pemulihan), dan $t_{rci} - t_d$ (waktu identifikasi akar masalah).

2. Untuk Lingkungan A (Manual): *Operator* menjalankan `recovery-runbook.sh` yang memandu pemulihan berdasarkan jenis penyimpangan. `metrics-collect.sh` mencatat waktu eksekusi setiap perintah korektif. Catat: (a) waktu *operator* mendeteksi deviasi (deteksi); (b) waktu *operator* menyelesaikan seluruh perintah korektif (pemulihan); (c) waktu *operator* mengidentifikasi penyebab penyimpangan melalui pemeriksaan log dan `kubectl describe` (identifikasi akar masalah). Hitung $t_d - t_0$, $t_2 - t_0$, dan $t_{rci} - t_d$.
3. Catat jumlah perintah `kubectl` yang diperlukan sebagai pengamatan pelengkap, direkam otomatis oleh `recovery-runbook.sh`.
4. Dokumentasikan *traceability*: catat kelengkapan *audit log* untuk Lingkungan B (Git *commit history* + ArgoCD *notifications*) dan Lingkungan A (Kubernetes *audit logs*).

3.12 Metode Pengujian (Eksperimen E1 s.d. E3)

3.12.1 E1: Konsistensi Deployment (R1)

Tabel 3.2: Desain Eksperimen E1: Konsistensi Deployment

Parameter	Deskripsi
Metrik	Persentase sumber daya yang cocok dengan <i>desired declarative state</i>
Cara Ukur	Hitung jumlah sumber daya <i>Healthy/Synced</i> dibagi total <i>expected resources</i>
Paradigma	(a) Manual/ <code>kubectl</code> , (b) GitOps/ArgoCD
Pengulangan	$n = 5$ per skenario per lingkungan

3.12.2 E2: Waktu Deteksi, Pemulihan, dan Identifikasi Akar Masalah (R2)

Tabel 3.3: Desain Eksperimen E2: Waktu Deteksi, Pemulihan, dan Identifikasi Akar Masalah

Parameter	Deskripsi
Metrik	Waktu deteksi ($t_d - t_0$), Waktu pemulihan ($t_2 - t_0$), Waktu identifikasi akar masalah ($t_{rci} - t_d$)
Cara Ukur	Lingkungan B: ArgoCD API <i>polling</i> setiap 5 detik; Lingkungan A: perintah <code>kubectl + operator runbook</code>
Paradigma	(a) Manual/ <code>kubectl</code> , (b) GitOps/ArgoCD (<i>self-heal</i> aktif)
Pengulangan	$n = 10$ per skenario per lingkungan

3.12.3 E3: *Traceability* Operasional (R3, Deskriptif)

Tabel 3.4: Desain Eksperimen E3: *Traceability* Operasional (Deskriptif)

Parameter	Deskripsi
Metrik	Kelengkapan <i>audit log</i> (%), kemudahan identifikasi akar masalah (deskriptif)
Cara Ukur	Lingkungan B: persentase perubahan yang dapat direkonstruksi dari Git <i>commit history</i> ; Lingkungan A: persentase perubahan yang dapat direkonstruksi dari Kubernetes <i>audit logs</i>
Paradigma	(a) Manual/ <code>kubectl</code> , (b) GitOps/ArgoCD
Catatan	Metrik ini dinilai secara deskriptif, bukan diuji secara statistik

3.13 Analisis Statistik Komparatif

Analisis ini bersifat eksploratif-komparatif, bukan konfirmatoris. Tujuannya adalah mengkarakterisasi perbedaan antara kedua paradigma *deployment* dan mengukur besarnya efek yang teramati, bukan menguji hipotesis nol secara formal.

Setelah pengumpulan data selesai, analisis statistik dilakukan dengan langkah berikut:

1. Statistik Deskriptif: Hitung *mean*, median, rentang, dan deviasi standar untuk

setiap metrik per lingkungan per skenario. Presentasikan dalam bentuk *box plot* per skenario dan per metrik.

2. Uji Perbandingan: Terapkan *Mann–Whitney U test* untuk membandingkan setiap metrik antara Lingkungan A dan Lingkungan B. Uji tersebut dipilih karena tidak mengasumsikan distribusi normal dan cocok untuk ukuran sampel yang digunakan. Tingkat signifikansi ditetapkan pada $\alpha = 0,05$.
3. Ukuran Efek: Hitung Cohen's d sebagai ukuran efek standar yang menunjukkan besarnya perbedaan praktis antara kedua kelompok. Interpretasi mengikuti konvensi Cohen: $|d| < 0,2$ (efek kecil), $|d| \approx 0,5$ (efek sedang), $|d| > 0,8$ (efek besar).

Metrik R3 (*traceability*) dinilai secara deskriptif dan tidak diuji secara statistik.

3.14 Ancaman terhadap Validitas

Berikut adalah identifikasi ancaman terhadap validitas beserta strategi mitigasi yang diterapkan, mengikuti kerangka Wohlin et al. (2012) dan Cook and Campbell (1979).

3.14.1 Validitas Internal

1. Seleksi (*Selection Bias*): Kedua lingkungan tidak ditugaskan secara acak dari populasi, melainkan peneliti membangun keduanya secara deliberatif. Mitigasi: Lingkungan A dan B dibangun pada klaster identik dengan konfigurasi yang disamakan; satu-satunya perbedaan adalah paradigma *deployment* (variabel independen).
2. Operator Tunggal (*Single Operator Bias*): Peneliti bertindak sebagai satu-satunya *operator* pada Lingkungan A, yang dapat membatasi generalisasi. Mitigasi: (a) *Operator* mengikuti *recovery runbook* yang terstandarisasi, sehingga keputusan subjektif diminimalkan; (b) seluruh *timestamp* direkam oleh skrip otomatis, bukan oleh manusia; (c) metrik yang tidak bergantung pada *operator* (R1 konsistensi, R2 deteksi otomatis ArgoCD) memberikan pengukuran objektif; (d) batasan ini diakui secara eksplisit dan direkomendasikan untuk replikasi dengan *operator* berbeda.

3. *Maturasi (Maturation)*: Kinerja *operator* dapat membaik sepanjang eksperimen karena pembelajaran. Mitigasi: Urutan skenario diacak; data pembelajaran *operator* dicatat; *practice run* dilakukan sebelum pengumpulan data resmi.
4. *Instrumentasi*: Perubahan pada alat ukur selama eksperimen (mis. pembaruan ArgoCD). Mitigasi: Seluruh versi *software* (ArgoCD, Kubernetes, komponen beban uji) dikunci sebelum eksperimen dimulai dan tidak diperbarui selama pengumpulan data.
5. *Efek Pengujian (Testing)*: *Operator* mungkin berubah perilakunya karena sadar sedang diobservasi (*Hawthorne effect*). Mitigasi: *Operator* mengikuti *recovery runbook* terstandarisasi; waktu reaksi direkam oleh skrip, bukan oleh penilaian manusia.

3.14.2 Validitas Eksternal

1. *Kekhususan Konteks*: Hasil mungkin tidak generalisasi ke klaster berukuran besar, *workload* yang berbeda, atau tim *operator* yang lebih besar. Mitigasi: Dokumentasikan seluruh karakteristik klaster, *workload*, dan *operator* secara transparan; gunakan beban uji yang mewakili kelas *workload* komposit multi-service.
2. *Interaksi Seleksi dan Perlakuan*: Efek paradigma *deployment* mungkin spesifik untuk tingkat keahlian *operator* dalam penelitian. Mitigasi: Catat profil keahlian *operator*; rekomendasikan replikasi dengan tim yang berbeda.
3. *Operator Tunggal*: Penggunaan satu *operator* membatasi generalisasi ke konteks tim yang beragam. Mitigasi: Standarisasi prosedur melalui *runbook* dan skrip otomasi; dokumentasikan batasan secara eksplisit; rekomendasikan replikasi multi-*operator* sebagai penelitian lanjutan.

3.14.3 Validitas Konstruk

1. *Bias Operasi Tunggal (Mono-operation Bias)*: Menggunakan satu beban uji (instans kelas *workload* komposit multi-service) dan satu alat GitOps (ArgoCD). Mitigasi: Beban uji dipilih karena kompleksitas yang mewakili kelas *workload* komposit multi-service; ArgoCD merupakan implementasi GitOps yang paling banyak digunakan.

2. Bias Metode Tunggal (*Mono-method Bias*): Metrik R1 dan R2 diukur secara otomatis oleh skrip otomatisasi; R3 bersifat deskriptif dan didokumentasikan berdasarkan bukti objektif (log dan Git *history*). Mitigasi: Seluruh metrik kuantitatif menggunakan pengukuran objektif berbasis *timestamp*; R3 dideskripsikan berdasarkan data yang dapat diverifikasi secara independen.
3. Operasionalisasi Konstruk: Definisi “penyimpangan konfigurasi” dibatasi pada 7 skenario yang merepresentasikan kesalahan operasional umum. Mitigasi: Skenario dirancang berdasarkan taksonomi Zhang et al. (2026) dan praktik yang dilaporkan oleh Pohjola (2025); batasan ini diakui secara eksplisit.
4. Kelemahan Perbandingan (*Obvious Comparison*): Perbandingan antara paradigma manual dan GitOps dapat dipandang sebagai perbandingan yang hasilnya sudah intuitif — GitOps, dengan *automated reconciliation* dan *audit trail* bawaan, seharusnya lebih unggul. Namun, penelitian ini bukan dimaksudkan untuk membuktikan arah perbedaan, melainkan untuk mengkarakterisasi *magnitude* dan *kondisi* perbedaan tersebut. Tanpa evaluasi terkontrol, praktisi tidak memiliki informasi kuantitatif mengenai seberapa besar perbedaan pada setiap skenario penyimpangan, pada kondisi apa perbedaan tersebut paling signifikan, dan apakah terdapat skenario di mana perbedaan tersebut tidak bermakna secara praktis. Mitigasi: analisis ukuran efek (Cohen’s *d*) digunakan untuk mengkarakterisasi besarnya perbedaan, bukan hanya arahnya; temuan yang menunjukkan efek kecil pada skenario tertentu sama informatifnya dengan temuan yang menunjukkan efek besar.

3.14.4 Reliabilitas

1. Reliabilitas Pengukuran: Pengukuran waktu (R2) bergantung pada presisi pencatatan *timestamp*. Mitigasi: Semua *timestamp* direkam secara otomatis oleh skrip otomatisasi; akurasi diverifikasi dengan sinkronisasi jam (NTP).
2. Reliabilitas Replikasi: Pihak lain harus dapat mereplikasi eksperimen. Mitigasi: Protokol eksperimen didokumentasikan secara rinci; konfigurasi kluster, versi *software*, dan seluruh skrip otomatisasi (`drift-inject.sh`, `metrics-collect.sh`, `baseline-check.sh`, `recovery-runbook.sh`) akan dipublikasikan sebagai *artifact* pendamping.

BAB IV

JADWAL PENELITIAN

Penelitian direncanakan untuk dilaksanakan selama enam bulan, dari bulan Juni 2026 hingga November 2026. Jadwal penelitian disusun berdasarkan tahapan evaluasi empiris terkontrol: studi literatur, perancangan lingkungan, pengembangan skrip otomasi, kalibrasi *baseline*, pelaksanaan eksperimen, analisis statistik, dan penulisan laporan. Tahapan-tahapan tersebut dipersempit dan saling tumpang tindih untuk efisiensi waktu.

Tabel 4.1 menunjukkan rencana jadwal pelaksanaan penelitian berbasis bulan. Simbol ● menunjukkan bahwa kegiatan dilaksanakan pada bulan tersebut.

Tabel 4.1: Jadwal Penelitian (Gantt Chart)

Kegiatan	Bulan					
	Jun	Jul	Agu	Sep	Okt	Nov
Studi Literatur & Perancangan Lingkungan	●	●				
Implementasi ArgoCD + <i>Baseline</i> Manual		●	●			
Pengembangan Skrip Otomasi		●	●			
Kalibrasi <i>Baseline</i> & Uji Coba			●	●		
Eksperimen & Pengumpulan Data (E1–E3)				●	●	
Analisis Statistik & Interpretasi					●	●
Penulisan Laporan	●	●	●	●	●	●
Seminar / Sidang						●

4.1 Penjelasan Kegiatan

1. Studi Literatur dan Perancangan Lingkungan (Juni – Juli 2026)

Kegiatan ini mencakup pengumpulan dan analisis literatur mengenai penyimpanan konfigurasi pada Kubernetes, evaluasi empiris GitOps, metode eksperimental pada riset sistem, serta klasifikasi *workload* multi-service. Sumber literatur meliputi jurnal ilmiah (IEEE, ACM, Springer), dokumentasi resmi, buku teks, dan artikel teknis. Tahap ini juga mencakup perancangan arsitektur Lingkungan A (*deployment* manual/`kubectl`) dan Lingkungan B (*deployment* Gi-

tOps/ArgoCD), definisi variabel dan intervensi, pemilihan tujuh skenario penyimpangan, serta spesifikasi protokol pengukuran.

2. Implementasi ArgoCD dan *Baseline* Manual (Juli – Agustus 2026)

Tahap implementasi meliputi konfigurasi ArgoCD pada klaster, pembuatan repositori GitOps dengan manifest deklaratif, konfigurasi *baseline* manual (*workflow* `kubectl`), serta konfigurasi *logging* dan *monitoring* (Prometheus, Grafana, Kubernetes *audit logs*, ArgoCD *notifications*). Pada akhir fase ini, kedua lingkungan siap untuk eksperimen.

3. Pengembangan Skrip Otomasi (Juli – Agustus 2026)

Pengembangan empat skrip otomasi yang mendukung konsistensi prosedural eksperimen: (a) `drift-inject.sh` untuk injeksi skenario penyimpangan secara acak dengan pencatatan *timestamp*; (b) `metrics-collect.sh` untuk pengumpulan metrik secara berkala via ArgoCD API dan `kubectl`; (c) `baseline-check.sh` untuk verifikasi kondisi klaster sebelum setiap *run*; dan (d) `recovery-runbook.sh` untuk memandu *operator* pada Lingkungan A melalui prosedur pemulihan terstandarisasi. Skrip dikembangkan secara paralel dengan implementasi lingkungan.

4. Kalibrasi *Baseline* dan Uji Coba (Agustus – September 2026)

Sebelum eksperimen dimulai, fase kalibrasi dilakukan untuk memastikan validitas internal. Kegiatan meliputi: (a) *deployment* beban uji (*workload*) komposit multi-service pada kedua lingkungan; (b) verifikasi *baseline* bersih; (c) pengukuran awal konsistensi; (d) verifikasi penggunaan sumber daya *node* di bawah 70%; dan (e) uji coba awal untuk memvalidasi protokol pengukuran dan skrip otomasi, termasuk *practice run operator*.

5. Eksperimen dan Pengumpulan Data (September – Oktober 2026)

Pelaksanaan tiga eksperimen sesuai desain pada Bab III. E1: konsistensi *deployment* ($n = 5$ per skenario). E2: waktu deteksi, waktu pemulihan, dan waktu identifikasi akar masalah ($n = 10$ per skenario). E3: *traceability* operasional (deskriptif, dokumentasi kelengkapan *audit log* per skenario). Data dikumpulkan dalam bentuk *timestamp*, status aplikasi, dan log. Skenario diacak untuk setiap lingkungan untuk memitigasi *order effect*. Seluruh prosedur dijalankan menggunakan skrip otomasi untuk konsistensi.

6. Analisis Statistik dan Interpretasi (Oktober – November 2026)

Analisis data meliputi statistik deskriptif (*mean*, median, rentang, deviasi standar), uji perbandingan (*Mann–Whitney U test*), perhitungan ukuran efek (Cohen's *d*), dan interpretasi hasil. Pembahasan mencakup identifikasi skenario penyimpangan yang paling berdampak, faktor yang memengaruhi hasil, serta perumusan rekomendasi pemilihan paradigma *deployment* berbasis bukti empiris terkontrol.

7. Penulisan Laporan (Juni – November 2026)

Penulisan laporan dilakukan secara paralel sepanjang penelitian. Setiap tahap menghasilkan bab atau bagian yang sesuai. Draft proposal diselesaikan pada pertengahan periode, draft skripsi lengkap pada akhir penelitian.

8. Seminar / Sidang (November 2026)

Presentasi dan sidang tugas akhir di depan dosen penguji, mencakup paparan hasil eksperimen, analisis statistik, rekomendasi paradigma, dan kesimpulan.

DAFTAR PUSTAKA

- R. Aghili, Q. Qin, H. Li, and F. Khomh. Understanding web application workloads and their applications: Systematic literature review and characterization. In *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024.
- Argo Project. ArgoCD documentation, 2024. URL <https://argo-cd.readthedocs.io>.
- Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.
- Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.
- Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.
- Cloud Native Computing Foundation. Cncf cloud native definition v1.0, 2024. URL <https://www.cncf.io/about/who-we-are>.
- CNCF GitOps Working Group. Opengitops principles v1.0.0. <https://opengitops.dev>, 2021.
- Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.
- Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.
- M. D'Amore. Gitops and argocd: Continuous deployment and maintenance of a full stack application in a hybrid cloud kubernetes environment. Master's thesis, Politecnico di Torino, 2021.
- Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.
- Sandra G. Hart and Lowell E. Staveland. Development of nasa-tlx (task load index): Results of empirical and theoretical research. *Advances in Psychology*, 52:139–183, 1988.

- Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- Thomas A. Limoncelli, Strata R. Chalup, and Christina J. Hogan. *The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems*. Addison-Wesley Professional, 2nd edition, 2018.
- M. E. Matubber. Managing 5g kubernetes infrastructures using gitops principles. Master’s thesis, KTH Royal Institute of Technology, 2025.
- Kief Morris. *Infrastructure as Code: Dynamic Systems for the Cloud Age*. O’Reilly Media, 2nd edition, 2020.
- P. Kagganti Nataraja. A security-centric analysis of declarative and imperative deployment approaches in kubernetes-based application environments. Master’s thesis, National College of Ireland, 2025.
- E. Paavola. Managing multiple applications on kubernetes using gitops principles. Master’s thesis, Turku University of Applied Sciences, 2021.
- O. Pohjola. Mitigating configuration drift in infrastructure-as-code systems. Master’s thesis, Aalto University, 2025.
- A. Rahman, S. I. Shamim, D. B. Bose, et al. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2023.
- D. Schlette, P. Empl, M. Caselli, T. Schreck, and G. Pernul. Do you play it by the books? a study on incident response playbooks and influencing factors. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024.
- Michael L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4th edition, 2015.
- R. Shrestha and A. A. N. Ali. Configuration management in kubernetes environments: A gitops approach. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2024.
- H. Siddiqui, F. Khendek, and M. Toeroe. Microservices based architectures for iot systems—state-of-the-art review. *Internet of Things*, 2023.

- Ervina Utami and Hanif Al Fatta. Analysis on the use of declarative and pull-based deployment models on gitops using argo cd. In *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2021.
- Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, pages 1–17. ACM, 2015.
- Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2nd edition, 2012.
- Billy Yuen, Alexander Matyushentsev, Jesse Suen, and Thomas Ekenstam. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*. O'Reilly Media, 2021. ISBN 978-1492094877.
- Yue Zhang, Uchswas Paul, Marcelo d'Amorim, and Akond Rahman. Configuration defects in kubernetes. *IEEE Transactions on Software Engineering*, 2026. doi: 10.48550/arXiv.2512.05062.